

Introduction to Server-Side Scripting : PHP (PHP: Hypertext Preprocessor)

1. PHP란 ?
2. PHP 프로그래밍 언어
3. PHP Forms
4. PHP Advanced
5. MySQL Database
6. PHP XML Parser

http://www.w3schools.com/php/default.asp

서강대학교
Sogang University

- **Market Share (24 Aug. 2026, https://w3techs.com/technologies)**

Most popular server-side programming languages

© W3Techs.com	usage	change since 1 July 2026
1. PHP	70.2%	-0.6%
2. JavaScript	7.2%	+0.5%
3. Ruby	7.0%	+0.2%
4. Java	5.4%	-0.1%
5. Scala	5.1%	+0.1%

percentages of sites

Most popular client-side programming languages

© W3Techs.com	usage	change since 1 July 2026
1. JavaScript	98.9%	
2. Flash	2.8%	

percentages of sites

Most popular JavaScript libraries

© W3Techs.com	usage	change since 1 July 2026	market share	change since 1 July 2026
1. jQuery	66.5%	-1.5%	86.5%	-0.8%
2. Bootstrap	15.1%	-0.4%	19.7%	-0.2%
3. Underscore	7.1%	-0.2%	9.3%	-0.1%
4. React	6.1%		7.9%	+0.1%
5. Modernizr	5.2%	-0.2%	6.7%	-0.2%

percentages of sites

Most popular web servers

© W3Techs.com	usage	change since 1 July 2026
1. Nginx	31.4%	-0.4%
2. Cloudflare Server	29.6%	+1.1%
3. Apache	22.6%	-0.6%
4. LiteSpeed	14.9%	-0.2%
5. Node.js	7.0%	+0.4%

percentages of sites

Most popular operating systems

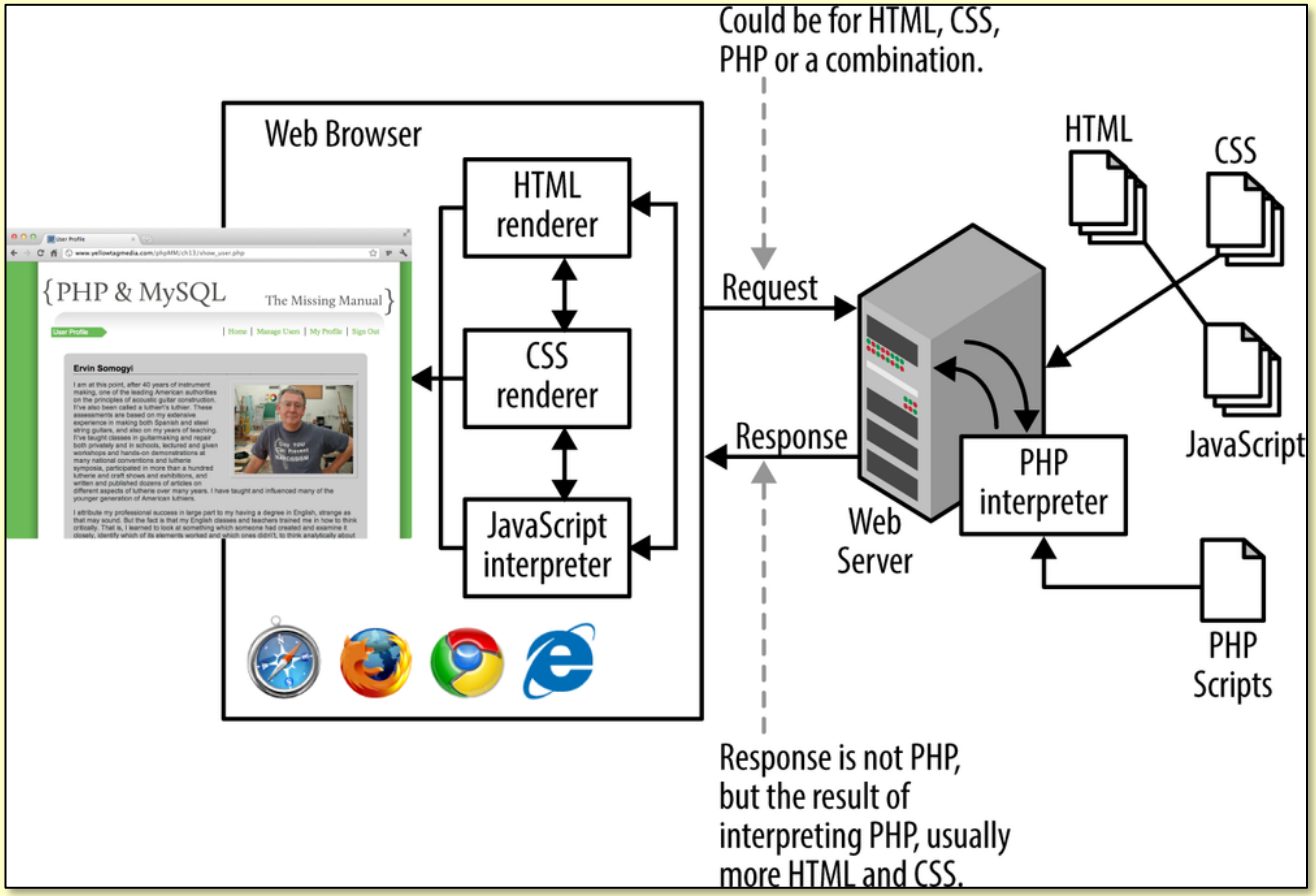
© W3Techs.com	usage	change since 1 July 2026
1. Unix	92.0%	+0.3%
2. Windows	8.2%	-0.3%

percentages of sites

- **Programming languages used in most popular websites**

https://en.wikipedia.org/wiki/Programming_languages_used_in_most_popular_websites

Websites	Popularity (unique visitors per month)[1]	Front-end (Client-side)	Back-end (Server-side)	Database
Google[2]	2,800,000,000	JavaScript, TypeScript	C, C++, Go,[3] Java, Python, Node	Bigtable,[4] MariaDB[5]
Facebook	1,120,000,000	JavaScript, TypeScript, Flow	Hack/HHVM, PHP, Python, C++, Java, Erlang, D,[6] Haskell[7]	MariaDB, MySQL,[8] HBase, Cassandra[9]
YouTube	1,100,000,000	JavaScript, TypeScript	Python, C, C++, Java,[10] Go[11]	Vitess, BigTable, MariaDB[5]
Yahoo	750,000,000	JavaScript	PHP	PostgreSQL, HBase, Cassandra, MongoDB,[12]
Etsy	516,000,000 (Total, not unique)[13]	JavaScript	PHP[14]	MySQL, Redis[15]
Amazon	2,400,000,000[16]	JavaScript	Java, C++, Perl[17]	DynamoDB, RDS/Aurora, Redshift[18]
Wikipedia	475,000,000	JavaScript	PHP	MariaDB[19]
Fandom	315,000,000[20]	JavaScript	PHP	MySQL
X	290,000,000	JavaScript	C++, Java,[21] Scala,[22] Ruby (Ruby on Rails)	MySQL[23]
Bing	285,000,000	JavaScript	C++, C#	Microsoft SQL Server, Cosmos DB
eBay	285,000,000	JavaScript	Java,[24] JavaScript,[25] Scala[26]	Oracle Database



https://ko.wikipedia.org/wiki/LAMP_(%EC%86%8C%ED%94%84%ED%8A%B8%EC%9B%A8%EC%96%B4_%EB%B2%88%EB%93%A4)

four open-source components

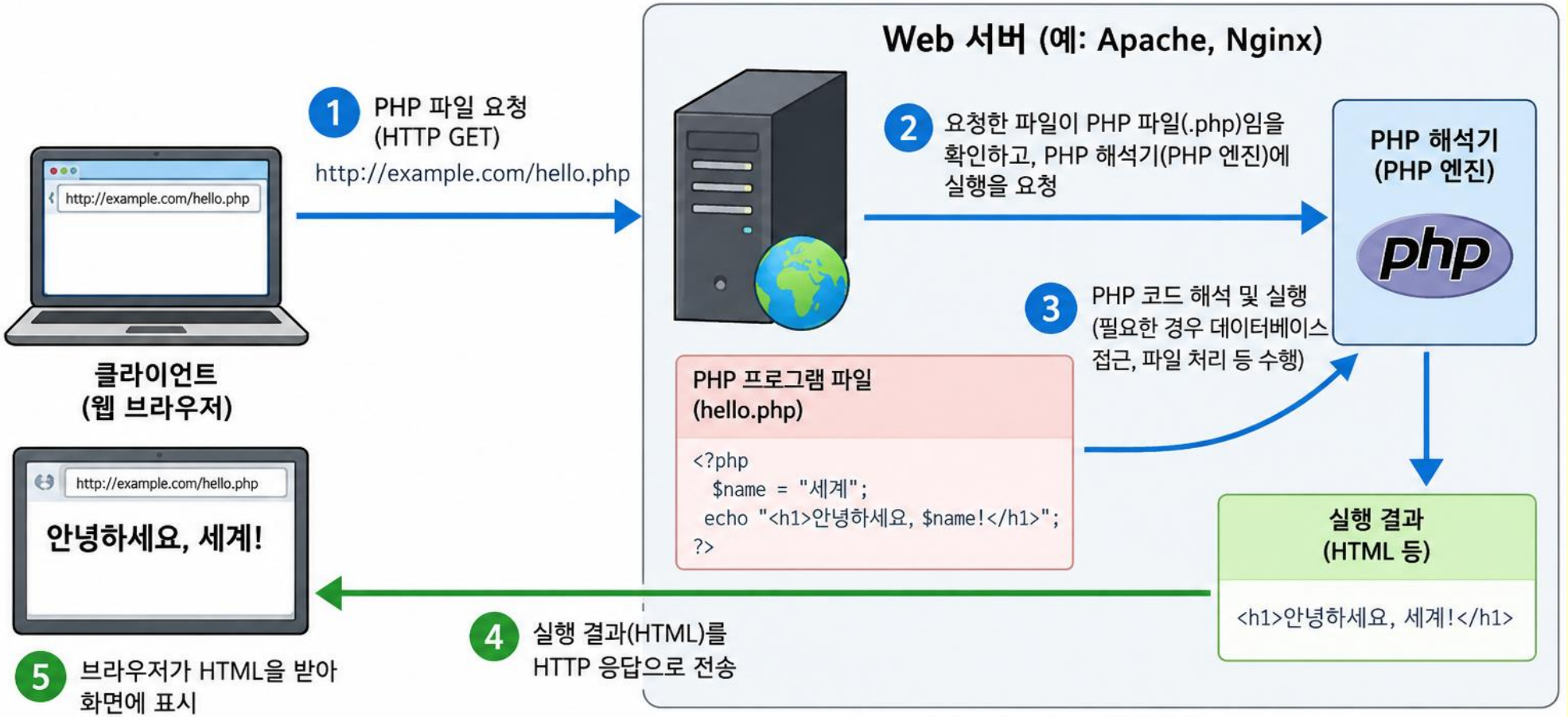


LAMP Stack : Web Service Stack

현재 저렴한 비용(모두 자유 소프트웨어이기 때문에 도입 비용은 없는 것이나 마찬가지이고 관리 비용만 쉽게 구할 수 있다는 것) 때문에 인기가 있다.

Web 서버에서 PHP 프로그램이 수행되는 과정

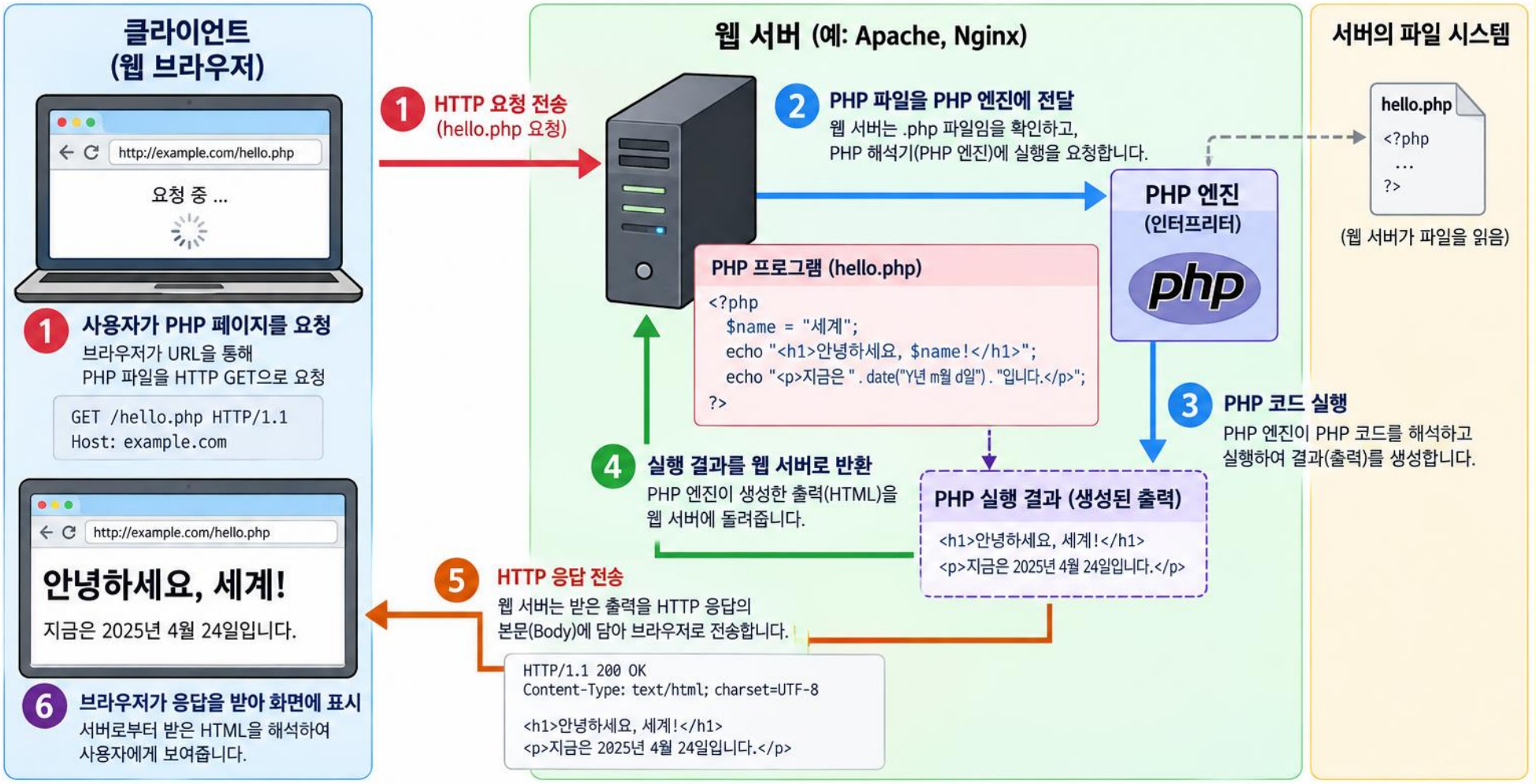
사용자가 브라우저에서 PHP 파일을 요청하면, Web 서버는 PHP 해석기를 통해 PHP 프로그램을 실행하고 생성된 결과(HTML)를 브라우저에 반환합니다.



요약: 사용자가 PHP 파일을 요청하면, Web 서버는 PHP 해석기를 통해 PHP 프로그램을 실행하고, 그 실행 결과로 생성된 HTML을 클라이언트(브라우저)에 반환합니다.

서버 측 PHP 프로그램의 출력이 웹 브라우저에 전달되는 과정

사용자가 브라우저에서 PHP 페이지를 요청하면, 웹 서버가 PHP 프로그램을 실행하고 그 실행 결과(주로 HTML)를 브라우저에게 응답합니다.



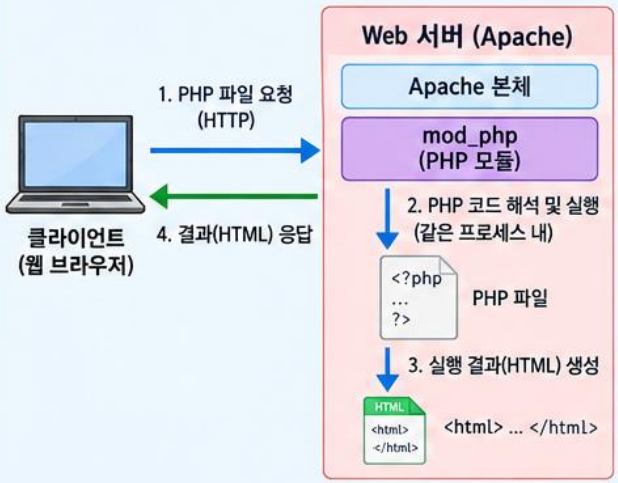
요약: 브라우저가 PHP 파일을 요청하면, 웹 서버가 PHP 엔진을 통해 PHP 프로그램을 실행하고, 그 실행 결과로 생성된 HTML을 HTTP 응답으로 브라우저에 전송하여 화면에 표시합니다.

PHP 실행 방식 비교 (mod_php, CGI, FastCGI)

웹 브라우저가 PHP 파일을 요청했을 때, 웹 서버가 PHP 코드를 어떻게 실행하는지 세 가지 방식의 구조와 특징을 비교합니다.

1. mod_php (Apache 모듈 방식)

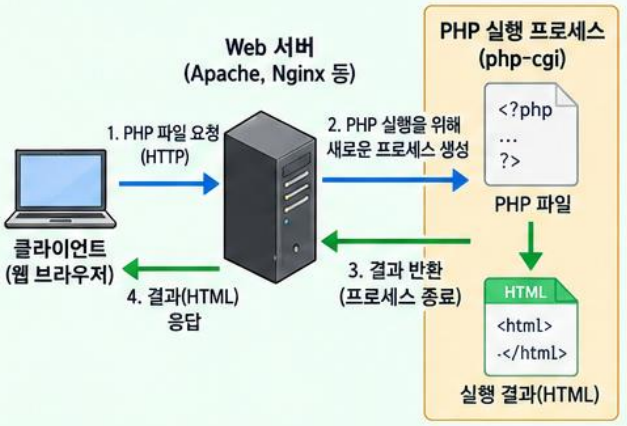
웹 서버 프로세스 안에 PHP 모듈이 내장되어 PHP 코드를 직접 실행



PHP가 Apache에 모듈로 내장되어 있어, 별도의 프로세스 없이 웹 서버 프로세스가 직접 PHP를 실행합니다.

2. CGI (Common Gateway Interface)

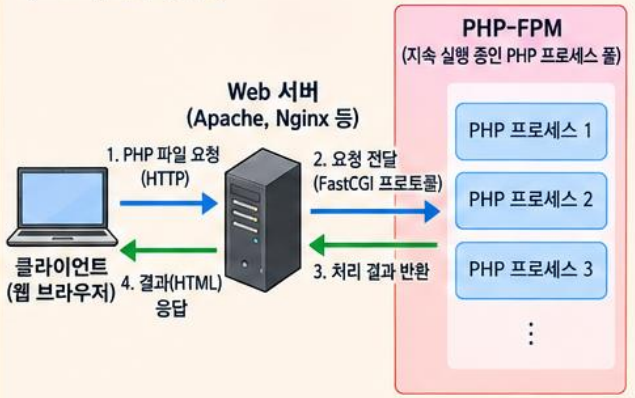
요청할 때마다 새로운 PHP 프로세스를 생성하여 실행



요청이 올 때마다 별도의 php-cgi 프로세스를 새로 생성하여 PHP 코드를 실행하고, 실행이 끝나면 프로세스를 종료합니다.

3. FastCGI (지속 실행 프로세스 방식)

PHP 프로세스를 미리 실행해 두고, 요청을 전달하여 처리 (프로세스를 재사용)



미리 실행해 둔 PHP 프로세스(PHP-FPM 등)가 요청을 처리하고, 프로세스를 재사용하여 빠르게 응답합니다.

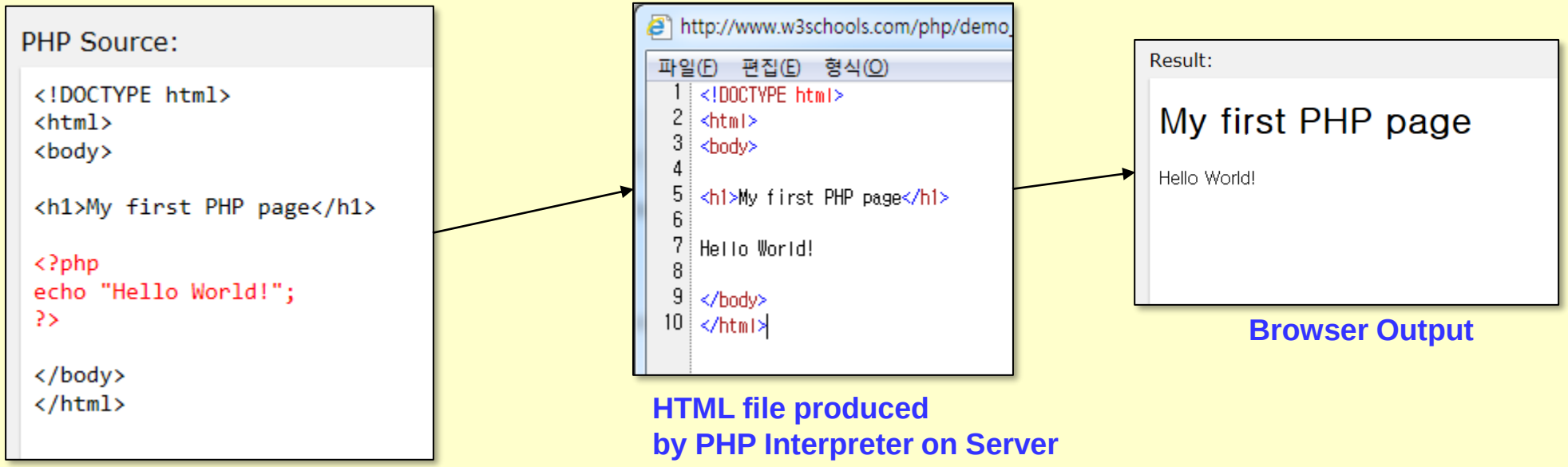
항목	mod_php	CGI	FastCGI (PHP-FPM 등)
실행 구조	웹 서버 프로세스 안에서 PHP 모듈로 실행	요청마다 새로운 PHP 프로세스 생성	미리 실행된 PHP 프로세스에 요청을 전달
프로세스 생성	추가 생성 없음 (웹 서버 프로세스 내)	요청마다 새 프로세스 생성 후 종료	미리 실행된 프로세스를 재사용 (지속 실행)
성능	매우 빠름 (프로세스 생성 오버헤드 없음)	느림 (매 요청마다 프로세스 생성)	빠름 (프로세스 재사용)
자원 사용	웹 서버 메모리 내에서 처리 (상대적으로 적음)	요청마다 프로세스 생성으로 자원 소모 큼	지속 프로세스로 효율적 (설정에 따라 조절 가능)
안정성 (격리성)	낮음 (웹 서버와 같은 프로세스)	높음 (독립 프로세스)	높음 (별도 프로세스)
주요 사용 예	과거 Apache에서 널리 사용 (현재는 권장되지 않음)	단순한 환경, 테스트, 특수한 경우	현재 가장 널리 사용 (PHP-FPM + Nginx/Apache)

요약: mod_php는 빠르지만 웹 서버와 결합되어 유연성이 낮고, CGI는 격리성이 높지만 매 요청마다 프로세스를 생성하여 느립니다. FastCGI는 미리 실행된 PHP 프로세스를 재사용하여 성능과 안정성을 모두 갖춘 방식으로, 현재 가장 많이 사용됩니다.

1. What is PHP ?

- PHP란 ?
  - PHP = PHP: Hypertext Preprocessor (What is the meaning of "Preprocessor" ?)
  - free to download and use
  - Server-side scripting language (executed on server) that may be embedded into HTML
  - Ultimate goal is to get PHP files to generate client-side code

    ⇔ must end up with HTML, CSS, JavaScript, other client-side code!



Source (test.php) on the Server

HTML file produced
by PHP Interpreter on Server

Browser Output

- **What is a PHP File?**
  - PHP files can contain text, HTML, CSS, JavaScript, and PHP code
  - PHP code are executed on the server, and the result is returned to the browser as plain HTML
  - PHP files have extension ".php"
- **What Can PHP Do?**
  - PHP can generate dynamic page content
  - PHP can create, open, read, write, delete, and close files on the server
  - PHP can collect form data
  - PHP can send and receive cookies
  - PHP can add, delete, modify data in your database
  - PHP can be used to control user-access
  - PHP can encrypt data
- **Why PHP?**
  - PHP runs on various platforms (Windows, Linux, Unix, Mac OS X, etc.)
  - PHP is compatible with almost all servers used today (Apache, IIS, etc.)
  - PHP supports a wide range of databases
  - PHP is free. Download it from the official PHP resource: www.php.net
  - PHP is easy to learn and runs efficiently on the server side

2. PHP as a Programming Language

- **Varaibles**
  - Store values for future reference, use variable name to refer to the value stored in it

```
$x = 42;        // store the value 42 in $x
echo $x;        // prints 42
echo $x+1;      // prints 43, value of $x is still 42
$x = 'hello!'   // type of $x can change
```

  - PHP is a un-typed language (dynamic typing)
    - ⇔ Do not need to declare the type of a variable
    - ⇔ Type can change throughout the program (by assignment)
  - all variable names are case-sensitive, and starts with the $ sign, followed by the name of the variable

```
$ php test.php
```

```
jhnang@cspro:~/public_html/cgi-bin$ php test.php
<!DOCTYPE html>
<html>
<body>
My car isred<br>PHP Notice:  Undefined variable: COLOR in /sogang/
blic_html/cgi-bin/test.php on line 7
My car is<br>PHP Notice:  Undefined variable: coLOR in /sogang/pro
c_html/cgi-bin/test.php on line 8
My car is<br>
</body>
</html>
```

PHP Source:

```
<!DOCTYPE html>
<html>
<body>

<?php
$color = "red";
echo "My car is " . $color . "<br>";
echo "My house is " . $COLOR . "<br>";
echo "My boat is " . $coLOR . "<br>";
?>

</body>
</html>
```

string concatenation

test.php

Result:

My car is red
My house is
My boat is

– PHP Variables Scope

⇔ variables can be declared anywhere in the script.

⇔ The scope of a variable is the part of the script where the variable can be referenced/used.

⇔ PHP has three different variable scopes:

⇒ local : variable declared within a function, can only be accessed within that function

⇒ global : variable declared outside a function, and can only be accessed outside a function (*not static scoping rule !!!*)

⇒ static : each time the function is called, that variable will still have the information it contained from the last time the function was called.

```php
<?php
$x = 5; // global scope

function myTest() {
    // using x inside this function will generate an error
    echo "<p>Variable x inside function is: $x</p>";
}
myTest();

echo "<p>Variable x outside function is: $x</p>";
?>
```

Result:

Variable x inside function is:

Variable x outside function is: 5

```php
<?php
function myTest() {
    $x = 5; // local scope
    echo "<p>Variable x inside function is: $x</p>";
}
myTest();

// using x outside the function will generate an error
echo "<p>Variable x outside function is: $x</p>";
?>
```

Result:

Variable x inside function is: 5

Variable x outside function is:

⇔ global, $GLOBAL[], static

PHP Source:

```
<!DOCTYPE html>
<html>
<body>

<?php
$x = 5;
$y = 10;

function myTest() {
     global $x, $y;
     $y = $x + $y;
}

myTest(); // run function
echo $y; // output the new value for variable $y
?>

</body>
</html>
```

Result:

15

PHP Source:

```
<!DOCTYPE html>
<html>
<body>

<?php
$x = 5;
$y = 10;

function myTest() {
     $GLOBALS['y'] = $GLOBALS['x'] + $GLOBALS['y'];
}
myTest();
echo $y;
?>

</body>
</html>
```

Result:

15

(built-in) associative array

PHP Source:

```
<!DOCTYPE html>
<html>
<body>

<?php
function myTest() {
     static $x = 0;
     echo $x;
     $x++;
}

myTest();
echo "<br>";
myTest();
echo "<br>";
myTest();
?>

</body>
</html>
```

Result:

0
1
2

static variable

- **Data Type**
  - PHP supports the following data types:
    - ⇔ String, Integer, Float (floating point numbers - also called double)
    - ⇔ Boolean, Array, Object, NULL, Resource

PHP Source:

```
<!DOCTYPE html>
<html>
<body>

<?php
class Car {
     function Car() {
         $this->model = "VW";
     }
}
// create an object
$herbie = new Car();

// show object properties
echo $herbie->model;
?>

</body>
</html>
```

Constructor
Data Member

Result:

VW

Object type

PHP Source:

```
<!DOCTYPE html>
<html>
<body>

<?php
$cars = array("Volvo","BMW","Toyota");
var_dump($cars);
?>

</body>
</html>
```

Result:

array(3) { [0]=> string(5) "Volvo" [1]=> string(3) "BMW" [2]=> string(6) "Toyota" }

PHP Source:

```
<!DOCTYPE html>
<html>
<body>

<?php
$x = 5985;
var_dump($x);
?>

</body>
</html>
```

Result:

int(5985)

– String type

```php
<?php
echo str_word_count("Hello world!"); // outputs 2
?>
```

```php
<?php
echo strrev("Hello world!"); // outputs !dlrow olleH
?>
```

```php
<?php
echo strpos("Hello world!", "world"); // outputs 6
?>
```

```php
<?php
echo str_replace("world", "Dolly", "Hello world!"); // outputs Hello Dolly!
?>
```

– Constant type

PHP Source:

```php
<!DOCTYPE html>
<html>
<body>

<?php
define("GREETING", "Welcome to W3Schools.com!");

function myTest() {
    echo GREETING;
}

myTest();
?>

</body>
</html>
```

Result:

Welcome to W3Schools.com!

Single and double quotes:

- Suppose `$str = 42;`
- `echo 'With single quotes, str is $str';`
  → output: `With single quotes, str is $str`
- `echo "With double quotes, str is $str";`
  → output: `With double quotes, str is 42`

• Operators

Operator	Name	Example
+	Addition	$x + $y
-	Subtraction	$x - $y
*	Multiplication	$x * $y
/	Division	$x / $y
%	Modulus	$x % $y
**	Exponentiation	$x ** $y

Assignment	Same as...
x = y	x = y
x += y	x = x + y
x -= y	x = x - y
x *= y	x = x * y
x /= y	x = x / y
x %= y	x = x % y

Operator	Name	Example	Result
==	Equal	$x == $y	Returns true if $x is equal to $y
===	Identical	$x === $y	Returns true if $x is equal to $y, and they are of the same type
!=	Not equal	$x != $y	Returns true if $x is not equal to $y
<>	Not equal	$x <> $y	Returns true if $x is not equal to $y
!==	Not identical	$x !== $y	Returns true if $x is not equal to $y, or they are not of the same type
>	Greater than	$x > $y	Returns true if $x is greater than $y
<	Less than	$x < $y	Returns true if $x is less than $y
>=	Greater than or equal to	$x >= $y	Returns true if $x is greater than or equal to $y
<=	Less than or equal to	$x <= $y	Returns true if $x is less than or equal to $y

PHP Source:

```
<!DOCTYPE html>
<html>
<body>

<?php
$x = 100;
$y = "100";

var_dump($x == $y); // returns true because values are equal
?>

</body>
</html>
```

PHP Source:

```
<!DOCTYPE html>
<html>
<body>

<?php
$x = 100;
$y = "100";

var_dump($x === $y); // returns false because types are not equal
?>

</body>
</html>
```

PHP Logical Operators

The PHP logical operators are used to combine conditional

Operator	Name	Example
and	And	$x and $y
or	Or	$x or $y
xor	Xor	$x xor $y
&&	And	$x && $y
||	Or	$x || $y
!	Not	!$x

PHP String Operators

PHP has two operators that are specially designed for strings.

Operator	Name	Example
.	Concatenation	$txt1 . $txt2
.=	Concatenation assignment	$txt1 .= $txt2

PHP Array Operators

The PHP array operators are used to compare arrays.

Operator	Name	Example	Result
+	Union	$x + $y	Union of $x and $y
==	Equality	$x == $y	Returns true if $x and $y have the same key/value pairs
===	Identity	$x === $y	Returns true if $x and $y have the same key/value pairs in the same order and of the same types
!=	Inequality	$x != $y	Returns true if $x is not equal to $y
<>	Inequality	$x <> $y	Returns true if $x is not equal to $y
!==	Non-identity	$x !== $y	Returns true if $x is not identical to $y

• if...else...elseif, switch Statements

```php
<?php
$t = date("H");

if ($t < "20") {
    echo "Have a good day!";
}
?>
```

```php
<?php
$t = date("H");

if ($t < "20") {
    echo "Have a good day!";
} else {
    echo "Have a good night!";
}
?>
```

```php
<?php
$t = date("H");

if ($t < "10") {
    echo "Have a good morning!";
} elseif ($t < "20") {
    echo "Have a good day!";
} else {
    echo "Have a good night!";
}
?>
```

```php
<?php
$favcolor = "red";

switch ($favcolor) {
    case "red":
        echo "Your favorite color is red!";
        break;
    case "blue":
        echo "Your favorite color is blue!";
        break;
    case "green":
        echo "Your favorite color is green!";
        break;
    default:
        echo "Your favorite color is neither red, blue, nor green!";
}
?>
```

`date(format,timestamp);`

Parameter	Description
format	Required. Specifies the format of the outputted date string. The following characters can be used: • d - The day of the month (from 01 to 31) • D - A textual representation of a day (three letters) • j - The day of the month without leading zeros (1 to 31) • l (lowercase 'L') - A full textual representation of a day • N - The ISO-8601 numeric representation of a day (1 for Monday, 7 for Sunday) • S - The English ordinal suffix for the day of the month (2 characters st, nd, rd or th. Works well with j) • w - A numeric representation of the day (0 for Sunday, 6 for Saturday) • z - The day of the year (from 0 through 365) • W - The ISO-8601 week number of year (weeks starting on Monday) • F - A full textual representation of a month (January through December) • m - A numeric representation of a month (from 01 to 12) • M - A short textual representation of a month (three letters) • n - A numeric representation of a month, without leading zeros (1 to 12) • t - The number of days in the given month • L - Whether it's a leap year (1 if it is a leap year, 0 otherwise) • o - The ISO-8601 year number • Y - A four digit representation of a year • y - A two digit representation of a year • a - Lowercase am or pm • A - Uppercase AM or PM • B - Swatch Internet time (000 to 999) • g - 12-hour format of an hour (1 to 12) • G - 24-hour format of an hour (0 to 23) • h - 12-hour format of an hour (01 to 12) • H - 24-hour format of an hour (00 to 23) • i - Minutes with leading zeros (00 to 59) • s - Seconds, with leading zeros (00 to 59) • u - Microseconds (added in PHP 5.2.2) • e - The timezone identifier (Examples: UTC, GMT, Atlantic/Azores) • I (capital i) - Whether the date is in daylights savings time (1 if Daylight Savings Time, 0 otherwise) • O - Difference to Greenwich time (GMT) in hours (Example: +0100)

• Loops

```php
<?php
$x = 1;

while($x <= 5) {
    echo "The number is: $x <br>";
    $x++;
}
?>
```

```php
<?php
$x = 1;

do {
    echo "The number is: $x <br>";
    $x++;
} while ($x <= 5);
?>
```

```php
<?php
for ($x = 0; $x <= 10; $x++) {
    echo "The number is: $x <br>";
}
?>
```

- **foreach** : for every loop iteration, the value of the current array element is assigned to $value and the array pointer is moved by one, until it reaches the last array element.

PHP Source:

```php
<!DOCTYPE html>
<html>
<body>

<?php
$colors = array("red", "green", "blue", "yellow");

foreach ($colors as $value) {
   echo "$value <br>";
}
?>

</body>
</html>
```

Result:

red
green
blue
yellow

- **Function**
  - a block of statements that can be used repeatedly in a program.
  - will not execute immediately when a page loads.
  - will be executed by a call to the function.

Default arguments

PHP Source:

```
<!DOCTYPE html>
<html>
<body>

<?php
function setHeight($minheight = 50) {
     echo "The height is : $minheight <br>";
}

setHeight(350);
setHeight();
setHeight(135);
setHeight(80);
?>

</body>
</html>
```

Result:

```
The height is : 350
The height is : 50
The height is : 135
The height is : 80
```

Return value

PHP Source:

```
<!DOCTYPE html>
<html>
<body>

<?php
function sum($x, $y) {
     $z = $x + $y;
     return $z;
}

echo "5 + 10 = " . sum(5,10) . "<br>";
echo "7 + 13 = " . sum(7,13) . "<br>";
echo "2 + 4 = " . sum(2,4);
?>

</body>
</html>
```

Result:

```
5 + 10 = 15
7 + 13 = 20
2 + 4 = 6
```

- **Array**
  - an array stores multiple values in one single variable:
  - In PHP, there are three types of arrays:
    - ⇔ Indexed arrays - Arrays with a numeric index
    - ⇔ Associative arrays - Arrays with named keys
    - ⇔ Multidimensional arrays - Arrays containing one or more arrays

Loop Through an Indexed Array

To loop through and print all the values of an indexed array, y

Example

```
<?php
$cars = array("Volvo", "BMW", "Toyota");
$arrlength = count($cars);

for($x = 0; $x < $arrlength; $x++) {
    echo $cars[$x];
    echo "<br>";
}
?>
```

Associative arrays

Example

```
<?php
$age = array("Peter"=>"35", "Ben"=>"37", "Joe"=>"43");
echo "Peter is " . $age['Peter'] . " years old.";
?>
```

```
<?php
$age = array("Peter"=>"35", "Ben"=>"37", "Joe"=>"43");

foreach($age as $x => $x_value) {
    echo "Key=" . $x . ", Value=" . $x_value;
    echo "<br>";
}
?>
```

Result:

Key=Peter, Value=35
Key=Ben, Value=37
Key=Joe, Value=43

– Multidimensional Arrays

PHP Source:

```
<!DOCTYPE html>
<html>
<body>

<?php
$cars = array
   (
   array("Volvo",22,18),
   array("BMW",15,13),
   array("Saab",5,2),
   array("Land Rover",17,15)
   );

for ($row = 0; $row <  4; $row++) {
   echo "<p><b>Row number $row</b></p>";
   echo "<ul>";
   for ($col = 0; $col <  3; $col++) {
     echo "<li>".$cars[$row][$col]."</li>";
   }
   echo "</ul>";
}
?>

</body>
</html>
```

Result:

Row number 0

- Volvo
- 22
- 18

Row number 1

- BMW
- 15
- 13

Row number 2

- Saab
- 5
- 2

Row number 3

- Land Rover
- 17
- 15

- Homogeneous Array *vs.* Heterogeneous Array

– Sorting array

⇔ sort(): sort arrays in ascending order
⇔ rsort(): sort arrays in descending order
⇔ asort(): sort *associative arrays* in ascending order, according to the value
⇔ ksort(): sort *associative arrays* in ascending order, according to the key
⇔ arsort(): sort *associative arrays* in descending order, according to the value
⇔ krsort() : sort *associative arrays* in descending order, according to the key

PHP Source:

```
<!DOCTYPE html>
<html>
<body>

<?php
$age = array("Peter"=>"35", "Ben"=>"37", "Joe"=>"43");
krsort($age);

foreach($age as $x => $x_value) {
     echo "Key=" . $x . ", Value=" . $x_value;
     echo "<br>";
}
?>

</body>
</html>
```

Result:

Key=Peter, Value=35
Key=Joe, Value=43
Key=Ben, Value=37

- **Superglobal variables**
  - Several predefined variables in PHP are "superglobals", which means that they are always accessible, regardless of scope
    - ⇔ can access them from any function, class or file without having to do anything special.
  - The PHP superglobal variables are:
    - ⇔ $GLOBALS : is used to access global variables from anywhere in the PHP script (also from within functions or methods).
    - ⇔ $_SERVER : holds information about headers, paths, and script locations
    - ⇔ $_REQUEST : used to collect data after submitting an HTML form.
    - ⇔ $_POST : used to collect form data after submitting an HTML form with method="post". $_POST is also widely used to pass variables.
    - ⇔ $_GET : used to collect form data after submitting an HTML form with method="get".
    - ⇔ $_FILES :
    - ⇔ $_ENV :
    - ⇔ $_COOKIE :
    - ⇔ $_SESSION :

모두 Associate Array : (key, value) 배열

– $_SERVER

PHP Source:

```
<!DOCTYPE html>
<html>
<body>

<?php
echo $_SERVER['PHP_SELF'];
echo "<br>";
echo $_SERVER['SERVER_NAME'];
echo "<br>";
echo $_SERVER['HTTP_HOST'];
echo "<br>";
echo $_SERVER['HTTP_REFERER'];
echo "<br>";
echo $_SERVER['HTTP_USER_AGENT'];
echo "<br>";
echo $_SERVER['SCRIPT_NAME'];
?>

</body>
</html>
```

Result:

```
/php/demo_global_server.php
www.w3schools.com
www.w3schools.com
http://www.w3schools.com/php/showphp.asp?filename=demo_global_server
Mozilla/5.0 (Windows NT 6.1; WOW64; Trident/7.0; rv:11.0) like Gecko
/php/demo_global_server.php
```

~/public_html/cgi-bin/php_server.php

Element/Code	Description
$_SERVER['PHP_SELF']	Returns the filename of the currently executing script
$_SERVER['GATEWAY_INTERFACE']	Returns the version of the Common Gateway Interface (CGI) the server is using
$_SERVER['SERVER_ADDR']	Returns the IP address of the host server
$_SERVER['SERVER_NAME']	Returns the name of the host server (such as www.w3schools.com)
$_SERVER['SERVER_SOFTWARE']	Returns the server identification string (such as Apache/2.2.24)
$_SERVER['SERVER_PROTOCOL']	Returns the name and revision of the information protocol (such as HTTP/1.1)
$_SERVER['REQUEST_METHOD']	Returns the request method used to access the page (such as POST)
$_SERVER['REQUEST_TIME']	Returns the timestamp of the start of the request (such as 1377687496)
$_SERVER['QUERY_STRING']	Returns the query string if the page is accessed via a query string
$_SERVER['HTTP_ACCEPT']	Returns the Accept header from the current request
$_SERVER['HTTP_ACCEPT_CHARSET']	Returns the Accept_Charset header from the current request (such as utf-8,ISO-8859-1)
$_SERVER['HTTP_HOST']	Returns the Host header from the current request
$_SERVER['HTTP_REFERER']	Returns the complete URL of the current page (not reliable because not all user-agents support it)
$_SERVER['HTTPS']	Is the script queried through a secure HTTP protocol
$_SERVER['REMOTE_ADDR']	Returns the IP address from where the user is viewing the current page
$_SERVER['REMOTE_HOST']	Returns the Host name from where the user is viewing the current page
$_SERVER['REMOTE_PORT']	Returns the port being used on the user's machine to communicate with the web server

모두 Associate Array : (key, value) 배열

– $_REQUEST

/tmp/jhnang/cgi-bin/_request.php

PHP Source:

```
<!DOCTYPE html>
<html>
<body>

<form method="post" action="<?php echo $_SERVER['PHP_SELF'];?>">
   Name: <input type="text" name="fname">
   <input type="submit">
</form>

<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
     // collect value of input field
     $name = $_REQUEST['fname'];
     if (empty($name)) {
         echo "Name is empty";
     } else {
         echo $name;
     }
}
?>

</body>
</html>
```

Returns the filename of the currently executing script

Result:

Name: 쿼리 전송

sss

3. PHP Forms

- **GET vs. POST**
  - The PHP superglobals $_GET and $_POST are used to collect form-data.

http://cspro.sogang.ac.kr/~jhnang/input2026.html

```
<html>
<body>

<form action="welcome.php" method="post">
Name: <input type="text" name="name"><br>
E-mail: <input type="text" name="email"><br>
<input type="submit">
</form>

</body>
</html>
```

Result:

Name: 낭종호
E-mail: jhnang
쿼리 전송

http://cspro.sogang.ac.kr/~jhnang/cgi-bin/welcome.php

```
<html>
<body>

Welcome <?php echo $_POST["name"]; ?><br>
Your email address is: <?php echo $_POST["email"]; ?>

</body>
</html>
```

Result:

Welcome 낭종호
Your email address is: jhnang

– GET *vs.* POST

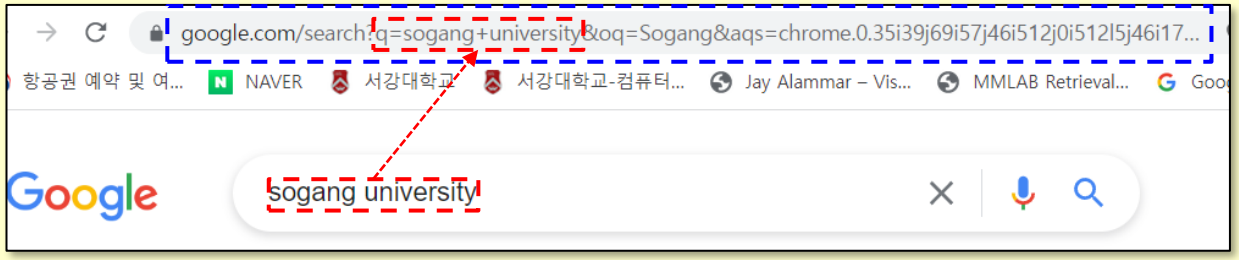
⇔ Both GET and POST create **an array** (e.g. **associative array**(key => value, key2 => value2, key3 => value3, ...)). This array holds key/value pairs, where keys are the names of the form controls and values are the input data from the user.

⇔ **GET :**

⇒ Information sent from a form with the GET method is ***visible to everyone*** (all variable names and values are displayed in the URL).

⇒ GET also has limits on the amount of information to send. The limitation is about *2000 characters*.

⇒ can be ***bookmarked (의미가 무엇일까??)***

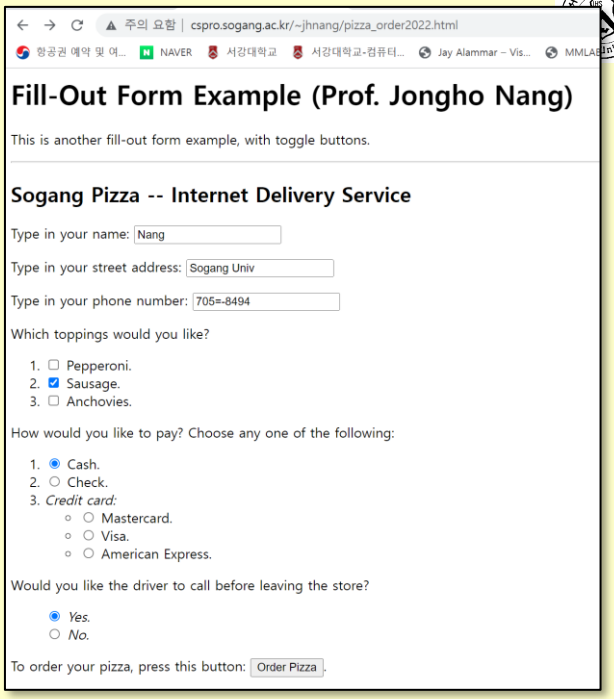


⇔ POST :

⇒ Information sent from a form with the POST method is **invisible to others** (all names/values are embedded within the body of the HTTP request)

⇒ has ***no limits*** on the amount of information to send.

⇒ supports advanced functionality such as support for ***multi-part binary input*** while uploading files to server.




```
nc -l -p 9090
POST /~jhnang/cgi-bin/pizza_order2022Echo.php HTTP/1.1
Host: cspro.sogang.ac.kr:9090
Connection: keep-alive
Content-Length: 79
Cache-Control: max-age=0
Upgrade-Insecure-Requests: 1
Origin: http://cspro.sogang.ac.kr
Content-Type: application/x-www-form-urlencoded
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chro
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/
change;v=b3;q=0.9
Referer: http://cspro.sogang.ac.kr/
Accept-Encoding: gzip, deflate
Accept-Language: ko-KR,ko;q=0.9,en-US;q=0.8,en;q=0.7
Cookie: _ga=GA1.3.1737228275.1653722538; Login=; _gid=GA1.3.150499508.1664157574

name=Nang&address=Sogang&phone=705&topping=sausage&paymethod=cash&callfirst=yes
```

② 브라우저가 submit버튼에 의하여 서버로 전송한 내용

(1) Form Example (No Validation) ➔ 실습

cgi_bin/form_validation.php

PHP Source:

```
<!DOCTYPE HTML>
<html>
<head>
</head>
<body>

<?php
// define variables and set to empty values
$name = $email = $gender = $comment = $website = "";

if ($_SERVER["REQUEST_METHOD"] == "POST") {
   $name = test_input($_POST["name"]);
   $email = test_input($_POST["email"]);
   $website = test_input($_POST["website"]);
   $comment = test_input($_POST["comment"]);
   $gender = test_input($_POST["gender"]);
}

function test_input($data) {
   $data = trim($data);
   $data = stripslashes($data);
   $data = htmlspecialchars($data);
   return $data;
}
?>

<h2>PHP Form Validation Example</h2>
<form method="post" action="<?php echo htmlspecialchars($_SERVER["PHP_SELF"]);?>">
   Name: <input type="text" name="name">
   <br><br>
   E-mail: <input type="text" name="email">
   <br><br>
   Website: <input type="text" name="website">
   <br><br>
   Comment: <textarea name="comment" rows="5" cols="40"></textarea>
   <br><br>
   Gender:
   <input type="radio" name="gender" value="female">Female
   <input type="radio" name="gender" value="male">Male
   <br><br>
   <input type="submit" name="submit" value="Submit">
</form>
```

앞뒤 blank 제거

따옴표 처리한 문자열을 풀어냄

문자열에서 특정한 특수 문자를 HTML 엔티티로 변환한다

????

특수 문자	변환된 문자
&(앰퍼샌드)	&
"(겹따옴표)	"
'(홑따옴표)	'
<(미만)	<
>(이상)	>

```
<?php
echo "<h2>Your Input:</h2>";
echo $name;
echo "<br>";
echo $email;
echo "<br>";
echo $website;
echo "<br>";
echo $comment;
echo "<br>";
echo $gender;
?>

</body>
```

PHP Form Validation Example

Name: 남종호

E-mail: jhnang@sogang.ac.kr

Website: mmlab.sogang.ac.kr

Comment: Test

Gender: ○ Female ◉ Male

Submit

Your Input:

Result:

PHP Form Validation Example

Name:

E-mail:

Website:

Comment:

Gender: ○ Female ○ Male

Submit

Your Input:

jhnang@sogang.ac.kr
mmlab.sogang.ac.kr
Test
male

http://cspro.sogang.ac.kr/~jhnang/cgi-bin/form_validationHTML5.php

4. PHP Advanced

- **include and require**
  - to insert the content of one PHP file into another PHP file (before the server executes it), with the **include** or **require** statement.
    - ⇔ **require** will produce a fatal error (E_COMPILE_ERROR) and stop the script
    - ⇔ **include** will only produce a warning (E_WARNING) and the script will continue

vars.php

```
<?php
$color='red';
$car='BMW';
?>
```

```
<html>
<body>

<h1>Welcome to my home page!</h1>
<?php include 'vars.php';
echo "I have a $color $car.";
?>

</body>
</html>
```

Result:

Welcome to my home page!

I have a red BMW.

- **File Handling**

  ⇔ PHP has several functions for creating, reading, uploading, and editing files.

  – readfile() Function

webdictionary.txt

```
AJAX = Asynchronous JavaScript and XML
CSS = Cascading Style Sheets
HTML = Hyper Text Markup Language
PHP = PHP Hypertext Preprocessor
SQL = Structured Query Language
SVG = Scalable Vector Graphics
XML = EXtensible Markup Language
```

PHP Source:

```
<!DOCTYPE html>
<html>
<body>

<?php
echo readfile("webdictionary.txt");
?>

</body>
</html>
```

Result:

AJAX = Asynchronous JavaScript and XML CSS = Cascading Style Sheets HTML = Hyper Text Markup Language PHP = PHP Hypertext Preprocessor SQL = Structured Query Language SVG = Scalable Vector Graphics XML = EXtensible Markup Language236

– Open/Read/Write File - `fopen()`, `fread()`, `fwrite()`

PHP Source:

```
<!DOCTYPE html>
<html>
<body>

<?php
$myfile = fopen("webdictionary.txt", "r") or die("Unable to open file!");
echo fread($myfile,filesize("webdictionary.txt"));
fclose($myfile);
?>

</body>
</html>
```

short-cut evalution

- `fgets()` : read a single line
- `fgetc()` : read a single character
- `feof()` : check "End-of-File"

`fopen()`

Modes	Description
r	**Open a file for read only**. File pointer starts at the beginning of the file
w	**Open a file for write only**. Erases the contents of the file or creates a new file if it doesn't exist. File pointer starts at the beginning of the file
a	**Open a file for write only**. The existing data in file is preserved. File pointer starts at the end of the file. Creates a new file if the file doesn't exist
x	**Creates a new file for write only**. Returns FALSE and an error if file already exists
r+	**Open a file for read/write**. File pointer starts at the beginning of the file
w+	**Open a file for read/write**. Erases the contents of the file or creates a new file if it doesn't exist. File pointer starts at the beginning of the file
a+	**Open a file for read/write**. The existing data in file is preserved. File pointer starts at the end of the file. Creates a new file if the file doesn't exist
x+	**Creates a new file for read/write**. Returns FALSE and an error if file already exists

- **Upload Files**
  - PHP is configured to allow file uploads.
    ⇔ In your "php.ini" file, search for the file_uploads directive, and set it to On:

"php.ini"

```
file_uploads = On
```

```
<!DOCTYPE html>
<html>
<body>

<form action="upload.php" method="post" enctype="multipart/form-data">
    Select image to upload:
    <input type="file" name="fileToUpload" id="fileToUpload">
    <input type="submit" value="Upload Image" name="submit">
</form>

</body>
</html>
```

- create an HTML form that allow users to choose the image file they want to upload:
- Make sure that the form uses method="**post**"
- The form also needs the following attribute: **enctype="multipart/form-data**". It specifies which content-type to use when submitting the form
- The type="**file**" attribute of the <input> tag shows the input field as a file-select control, with a "Browse" button next to the input control

upload.php

```php
<?php
$target_dir = "uploads/";
$target_file = $target_dir . basename($_FILES["fileToUpload"]["name"]);
$uploadOk = 1;
$imageFileType = pathinfo($target_file,PATHINFO_EXTENSION);
// Check if image file is a actual image or fake image
if(isset($_POST["submit"])) {
    $check = getimagesize($_FILES["fileToUpload"]["tmp_name"]);
    if($check !== false) {
        echo "File is an image - " . $check["mime"] . ".";
        $uploadOk = 1;
    } else {
        echo "File is not an image.";
        $uploadOk = 0;
    }
}
// Check if file already exists
if (file_exists($target_file)) {
    echo "Sorry, file already exists.";
    $uploadOk = 0;
}
// Check file size
if ($_FILES["fileToUpload"]["size"] > 500000) {
    echo "Sorry, your file is too large.";
    $uploadOk = 0;
}
```

```php
// Allow certain file formats
if($imageFileType != "jpg" && $imageFileType != "png" && $imageFileType != "jpeg"
&& $imageFileType != "gif" ) {
    echo "Sorry, only JPG, JPEG, PNG & GIF files are allowed.";
    $uploadOk = 0;
}
// Check if $uploadOk is set to 0 by an error
if ($uploadOk == 0) {
    echo "Sorry, your file was not uploaded.";
// if everything is ok, try to upload file
} else {
    if (move_uploaded_file($_FILES["fileToUpload"]["tmp_name"], $target_file)) {
        echo "The file ". basename( $_FILES["fileToUpload"]["name"]). " has been uploaded.";
    } else {
        echo "Sorry, there was an error uploading your file.";
    }
}
?>
```

- **Cookie**
  - A cookie is a small file that *the server embeds on the user's computer*.
  - Each time the same computer requests a page with a browser, it will send the cookie too.

The following example creates a cookie named "user" with the value "John Doe". The cookie will expire after 30 days (86400 * 30). The "/" means that the cookie is available in entire website

PHP Source:

```
<!DOCTYPE html>
<?php
$cookie_name = "user";
$cookie_value = "John Doe";
setcookie($cookie_name, $cookie_value, time() + (86400 * 30), "/"); // 86400 = 1
day
?>
<html>
<body>

<?php
if(!isset($_COOKIE[$cookie_name])) {
      echo "Cookie named '" . $cookie_name . "' is not set!";
} else {
      echo "Cookie '" . $cookie_name . "' is set!<br>";
      echo "Value is: " . $_COOKIE[$cookie_name];
}
?>

<p><strong>Note:</strong> You might have to reload the page to see
the cookie.</p>

</body>
</html>
```

To delete a cookie, use the `setcookie()` function ***with an expiration date in the past:***

Result: 1st load

Cookie named 'user' is not set!

Note: You might have to reload the page to see the value of the cookie.

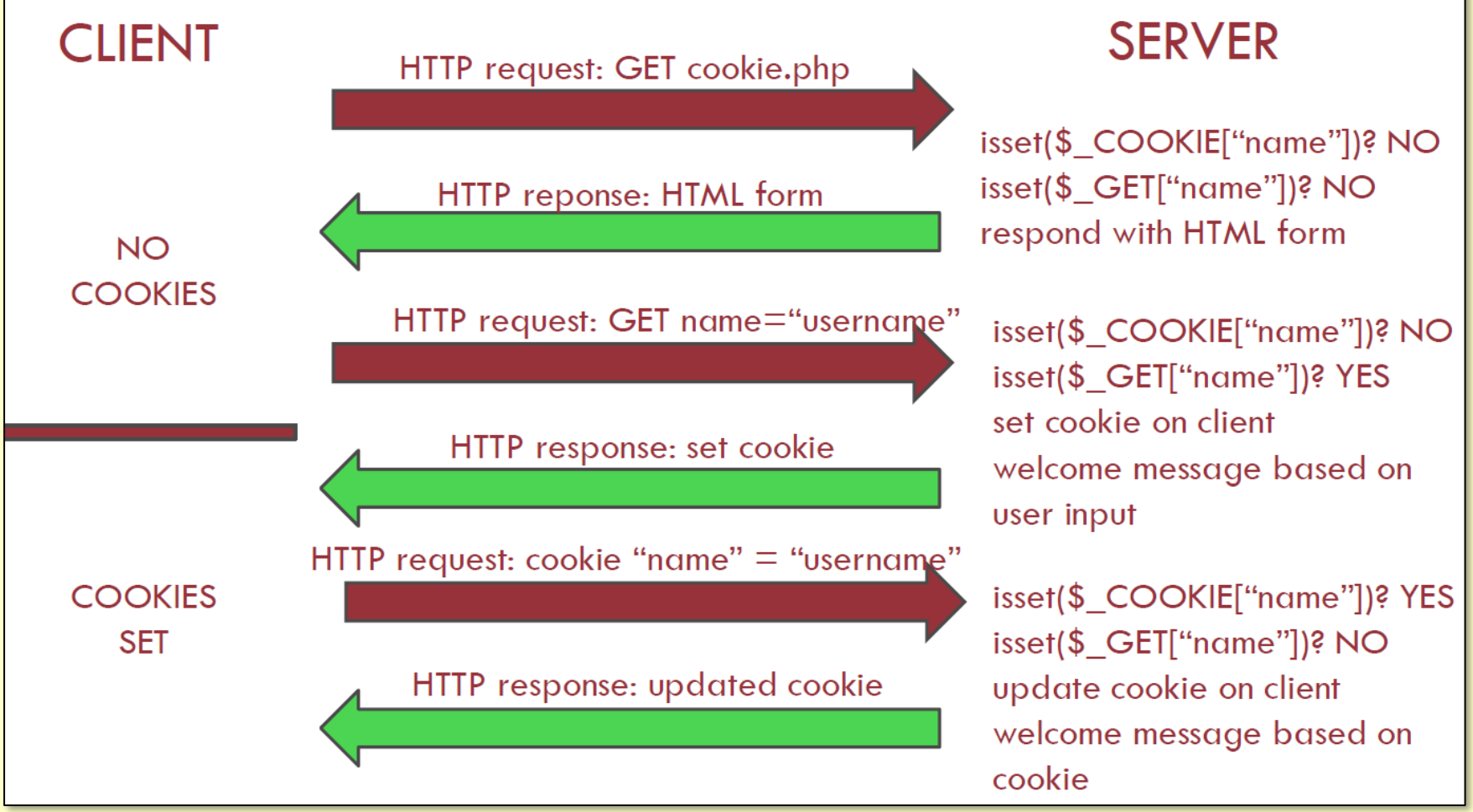
Result: 2nd load (reload)

Cookie 'user' is set!
Value is: John Doe

Note: You might have to reload the page to see the value of the cookie.

http://cspro.sogang.ac.kr/~jhnang/cgi-bin/set_cookie.php

Contents of the HTTP request/response



- **Session**
  - A session is a way to store information (in variables) to be used across multiple pages.
  - Unlike a cookie, the information is not stored on the users computer
  - Session variables stores user information to be used across multiple pages (e.g. username, favorite color, etc).
  - By default, session variables are valid *until the user closes the browser*.

set_sessionVar.php

PHP Source:

```php
<?php
// Start the session
session_start();
?>
<!DOCTYPE html>
<html>
<body>

<?php
// Set session variables
$_SESSION["favcolor"] = "green";
$_SESSION["favanimal"] = "cat";
echo "Session variables are set.";
?>

</body>
</html>
```

get_sessionVar.php

PHP Source:

```php
<?php
session_start();
?>
<!DOCTYPE html>
<html>
<body>

<?php
// Echo session variables that were set on previous page
echo "Favorite color is " . $_SESSION["favcolor"] . ".<br>";
echo "Favorite animal is " . $_SESSION["favanimal"] . ".";
?>

</body>
</html>
```

Result:

Favorite color is green.
Favorite animal is cat.

```php
<?php
// to change a session variable, just overwrite it
$_SESSION["favcolor"] = "yellow";
print_r($_SESSION);
?>
```

Modify session variable

```php
<?php
// remove all session variables
session_unset();

// destroy the session
session_destroy(`
?>
```

Destory PHP Session

– Cookie vs. Session

클라이언트에 브라우져를 2개 수행시키면 Session이 2개의 브라우져상에서 공유되나?? (같은 종류의 브라우져, 다른 종류의 브라우져) How ?

⇔ Two main disadvantages of cookies

⇒ Limited in size by browser

⇒ Stored client-side ➔ can be tampered with

⇔ Sessions store user data on the server

⇒ Limited only by server space

⇒ Cannot be modified by users

Web 서버는 클라이언트의 브라우져가 closed되었다는 것을 어떻게 알까?? ➔ 브라우저 종료와 서버 세션 만료는 서로 다른 사건

⇔ A potential downside to sessions is that they expire when the browser is closed

⇔ Sessions are identified by a session id: often a small cookie! But the rest of the data is still stored on the server

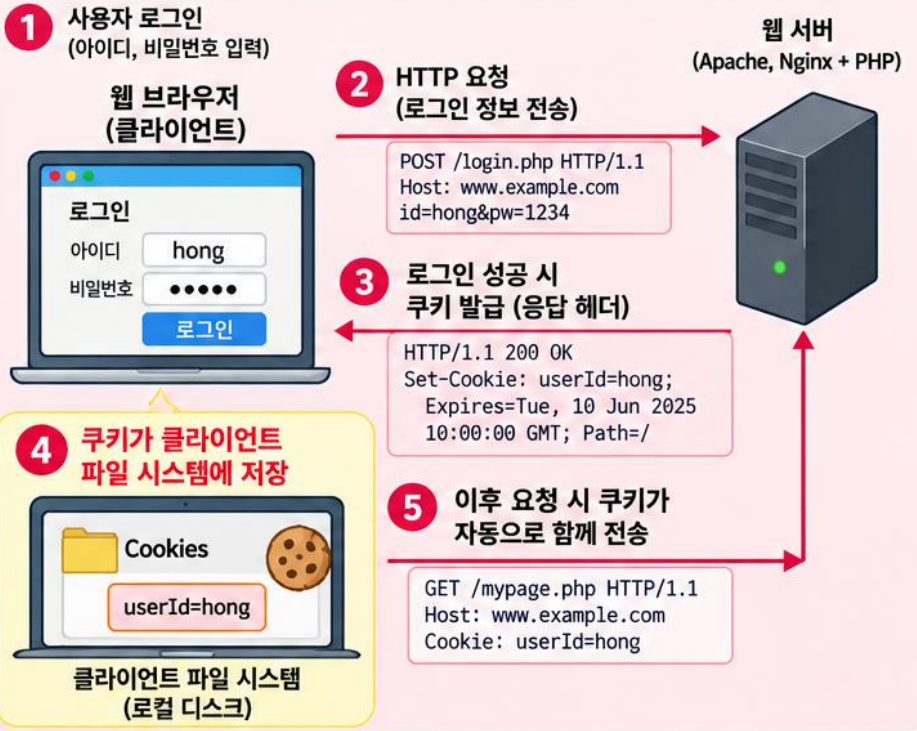
	COOKIES	SESSIONS
Where is data stored?	Locally on client	Remotely on server
Expiration?	Variable – determined when cookie is set	Session is destroyed when the browser is closed
Size limit?	Depends on browser	Depends only on server (practically no size limit)
Accessing information?	$_COOKIE	$_SESSION
General use?	Remember small things about the user, such as login name. Remember things after re-opening browser	Remembering varying amount of data about the user in one browsing "session"

- A session creates a file in a temporary directory on the server where registered session variables and their values are stored.
- A cookie called **PHPSESSID** is automatically sent to the user's computer to store unique session identification string
- When a PHP script wants to retrieve the value from a session variable, PHP automatically gets the unique session identifier string from the PHPSESSID cookie and then looks in its temporary directory for the file bearing that name and a validation can be done by comparing both values.
- A session ends when the user loses the browser or after leaving the site, the server will terminate the session after a predetermined period of time, commonly 30 minutes duration.

WWW 환경에서 상태(state) 저장 방법 : 쿠키(COOKIE)와 세션(SESSION) 비교

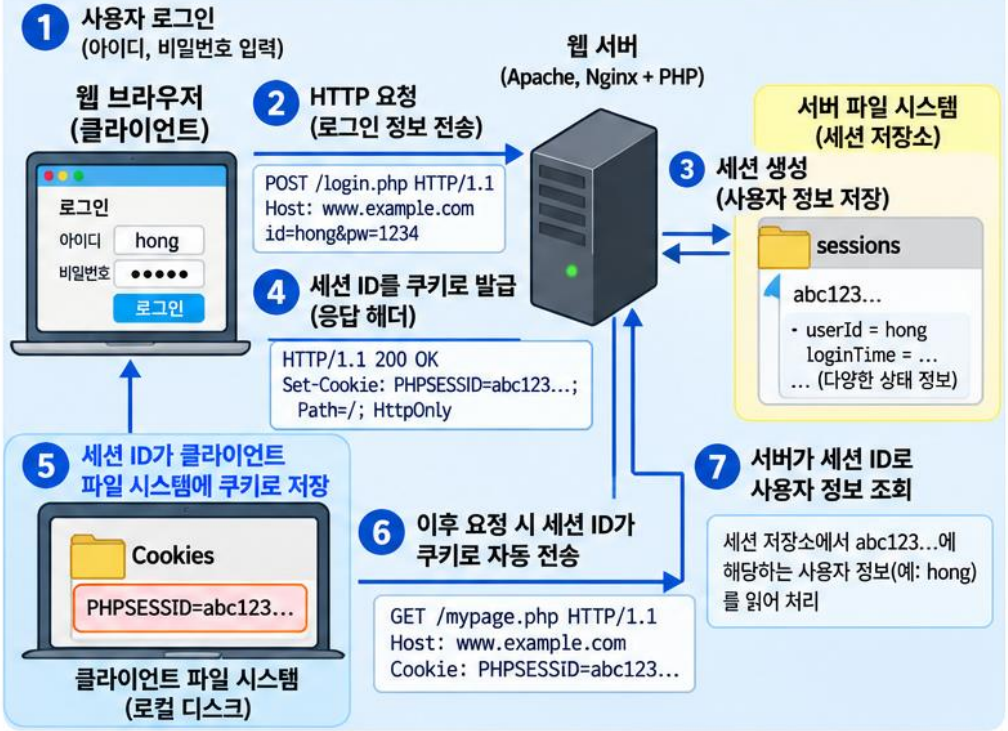
쿠키 (COOKIE)

사용자의 브라우저(클라이언트) 파일 시스템에 작은 데이터를 저장하여 요청마다 함께 전송



세션 (SESSION)

사용자별 상태 정보를 서버의 파일 시스템(세션 저장소)에 저장하고, 브라우저에는 세션을 식별하기 위한 세션 ID만 쿠키로 저장



구분	쿠키 (COOKIE)	세션 (SESSION)
저장 위치	클라이언트의 파일 시스템 (브라우저의 쿠키 저장 영역)	서버의 파일 시스템 (메모리 또는 파일, DB 등 세션 저장소)
저장 내용	사용자가 지정한 작은 데이터(문자열)	사용자의 상태 정보 전체 (객체, 배열 등 다양한 데이터)
브라우저 저장 예시	userId=hong	PHPSESSID=abc123... (세션 ID만 저장)
서버 저장 예시	저장하지 않음 (요청 시 브라우저에서 전송)	abc123... ⇒ { userId=hong, loginTime=..., 장바구니=..., ... }
데이터 크기	작음 (대략 4KB 이내)	상대적으로 큼 (제한 거의 없음, 서버 자원에 따라 다름)
보안성	클라이언트에 저장되어 위변조 가능 (중요한 정보 저장에 부적합)	서버에 저장되어 비교적 안전 (세션 탈취 방지를 위해 HTTPS, 세션 ID 관리 필요)
유지 기간	만료 시간(Expires/Max-Age) 설정 가능 (브라우저 종료 후에도 유지 가능)	기본적으로 브라우저 종료 시 만료 (설정에 따라 유지 가능)
주요 용도	간단한 사용자 설정, 자동 로그인, 방문 횟수 등	로그인 상태 유지, 사용자 정보 관리, 장바구니, 결제 과정 등 민감한 상태 관리

- **Filters**
  - are used to validate and sanitize external input.
  - Why ? : invalid submitted data can lead to security problems and break your webpage!
  - filter_var() : filters a single variable with a specified filter

PHP Source:

```
<!DOCTYPE html>
<html>
<body>

<?php
$str = "<h1>Hello World!</h1>";
$newstr = filter_var($str, FILTER_SANITIZE_STRING);
echo $newstr;
?>

</body>
</html>
```

Result:

Hello World!

Sanitize a String

PHP Source:

```
<!DOCTYPE html>
<html>
<body>

<?php
$ip = "127.0.0.1";

if (!filter_var($ip, FILTER_VALIDATE_IP) === false) {
     echo("$ip is a valid IP address");
} else {
     echo("$ip is not a valid IP address");
}
?>

</body>
</html>
```

Result:

127.0.0.1 is a valid IP address

Validate an IP Address

```
<?php
$email = "john.doe@example.com";

// Remove all illegal characters from email
$email = filter_var($email, FILTER_SANITIZE_EMAIL);

// Validate e-mail
if (!filter_var($email, FILTER_VALIDATE_EMAIL) === false) {
    echo("$email is a valid email address");
} else {
    echo("$email is not a valid email address");
}
?>
```

Sanitize and Validate an Email Address

위생 처리하다, 살균하다

```
<?php
$url = "http://www.w3schools.com";

// Remove all illegal characters from a url
$url = filter_var($url, FILTER_SANITIZE_URL);

// Validate url
if (!filter_var($url, FILTER_VALIDATE_URL) === false) {
    echo("$url is a valid URL");
} else {
    echo("$url is not a valid URL");
}
?>
```

Sanitize and Validate a URL

– Advaced filters

- HTML5 Tag를 사용하면 Self-Validation할 수 있는데, 이런 기능이 왜 필요할까?

⇔ Validate an Integer Within a Range

```
$int = 122;
$min = 1;
$max = 200;

if (filter_var($int, FILTER_VALIDATE_INT, array("options" => array("min_range"=>$min,
"max_range"=>$max))) === false) {
```

⇔ Validate IPv6 Address

```
$ip = "2001:0db8:85a3:08d3:1319:8a2e:0370:7334";

if (!filter_var($ip, FILTER_VALIDATE_IP, FILTER_FLAG_IPV6) === false) {
```

⇔ Validate URL - Must Contain QueryString

```
$url = "http://www.w3schools.com";

if (!filter_var($url, FILTER_VALIDATE_URL, FILTER_FLAG_QUERY_REQUIRED) === false) {
```

⇔ Remove Characters With ASCII Value > 127

```
$str = "<h1>Hello WorldÆØÅ!</h1>";

$newstr = filter_var($str, FILTER_SANITIZE_STRING, FILTER_FLAG_STRIP_HIGH);
```

5. MySQL Database

- With PHP, you can connect to and manipulate databases.
- MySQL is the most popular database system used with PHP.

- **What is MySQL?**
  - MySQL is a database system used on the web
  - MySQL is a database system that runs on a server
  - MySQL is ideal for both small and large applications
  - MySQL is very fast, reliable, and easy to use
  - MySQL uses standard SQL
  - MySQL compiles on a number of platforms
  - MySQL is free to download and use
  - MySQL is developed, distributed, and supported by Oracle Corporation
  - MySQL is named after co-founder Monty Widenius's daughter: My
  - The data in a MySQL database are stored in tables.
    ⇔ A table is a collection of related data, and it consists of columns and rows.
  - MySQL is the de-facto standard database system for web sites with HUGE volumes of both data and end-users (like Facebook, Twitter, and Wikipedia).

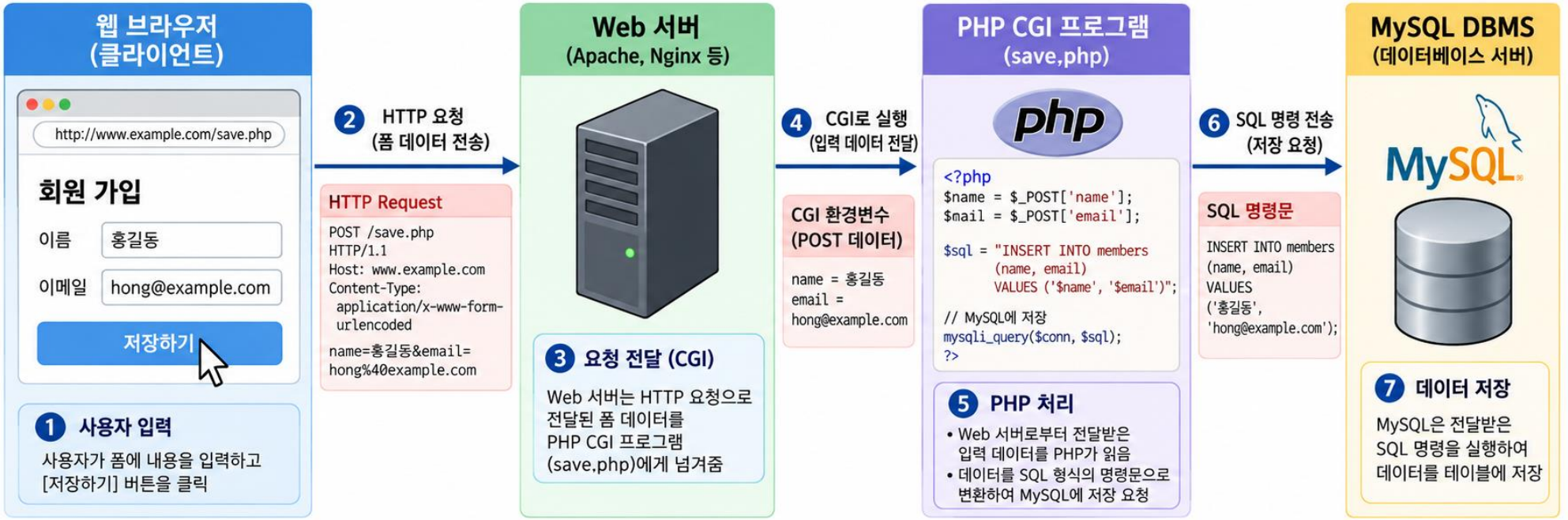
- **PHP + MySQL Database System**
  - PHP combined with MySQL are cross-platform (you can develop in Windows and serve on a Unix platform)

- **Database Queries**
  - A query is a question or a request.
  - We can query a database for specific information and have a record set returned.
  - Look at the following query (using standard SQL):

```
SELECT LastName FROM Employees
```

⇔ The query above selects all the data in the "LastName" column from the "Employees" table.

사용자 입력이 웹 서버, PHP, MySQL로 전달되어 저장되는 전체 과정



1. 사용자(브라우저)가 폼에 내용을 입력하고 [저장하기]를 클릭한다.
2. 브라우저는 입력한 내용을 HTTP 프로토콜(POST 방식)로 Web 서버에 전송한다.
3. Web 서버(Apache, Nginx 등)는 요청을 받아 PHP CGI 프로그램(save.php)에게 넘겨준다.
4. Web 서버는 입력 데이터를 CGI 환경변수(POST 데이터) 형태로 PHP에 전달한다.
5. PHP 프로그램은 전달받은 데이터를 읽어 SQL 명령문으로 변환한다.
6. PHP는 MySQL DBMS에 SQL 명령문(INSERT)을 전송하여 저장을 요청한다.
7. MySQL은 SQL을 실행하여 데이터를 데이터베이스 테이블에 저장한다.

- **PHP Connect to MySQL**
  - PHP 5 and later can work with a MySQL database using:
    - ⇔ MySQLi extension (the "i" stands for improved)
    - ⇔ PDO (PHP Data Objects)
  - **Open a Connection to MySQL**

Example (MySQLi Object-Oriented)

```php
<?php
$servername = "localhost";
$username = "username";
$password = "password";

// Create connection
$conn = new mysqli($servername, $username, $password);

// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
echo "Connected successfully";
?>
```

```php
$conn->close();
```

Example (MySQLi Procedural)

```php
<?php
$servername = "localhost";
$username = "username";
$password = "password";

// Create connection
$conn = mysqli_connect($servername, $username, $password);

// Check connection
if (!$conn) {
    die("Connection failed: " . mysqli_connect_error());
}
echo "Connected successfully";
?>
```

```php
mysqli_close($conn);
```

Example (PDO)

```php
<?php
$servername = "localhost";
$username = "username";
$password = "password";

try {
    $conn = new PDO("mysql:host=$servername;dbname=myDB", $username, $password);
    // set the PDO error mode to exception
    $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    echo "Connected successfully";
    }
catch(PDOException $e)
    {
    echo "Connection failed: " . $e->getMessage();
    }
?>
```

```php
$conn = null;
```

• Create a MySQL Database and Table

Example (MySQLi Object-oriented)

```php
<?php
$servername = "localhost";
$username = "username";
$password = "password";
$dbname = "myDB";

// Create connection
$conn = new mysqli($servername, $username, $password, $dbname);
// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

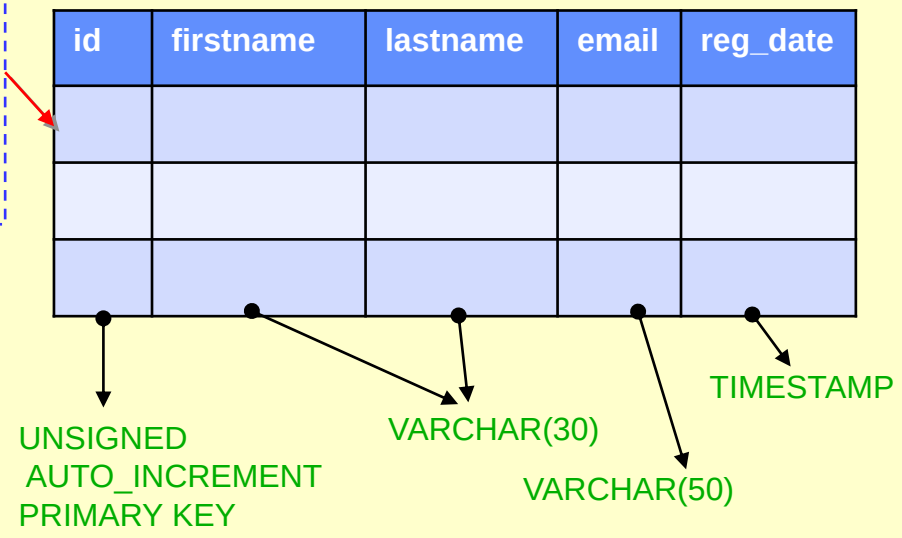
// sql to create table
$sql = "CREATE TABLE MyGuests (
id INT(6) UNSIGNED AUTO_INCREMENT PRIMARY KEY,
firstname VARCHAR(30) NOT NULL,
lastname VARCHAR(30) NOT NULL,
email VARCHAR(50),
reg_date TIMESTAMP
)";

if ($conn->query($sql) === TRUE) {
    echo "Table MyGuests created successfully";
} else {
    echo "Error creating table: " . $conn->error;
}

$conn->close();
?>
```

Connect to DB and
create a database named “myDB”

Create a table named “MyGuest”



• Insert Data Into MySQL

Example (MySQLi Object-oriented)

```
<?php
$servername = "localhost";
$username = "username";
$password = "password";
$dbname = "myDB";

// Create connection
$conn = new mysqli($servername, $username, $password, $dbname);
// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

$sql = "INSERT INTO MyGuests (firstname, lastname, email)
VALUES ('John', 'Doe', 'john@example.com')";

if ($conn->query($sql) === TRUE) {
    echo "New record created successfully";
} else {
    echo "Error: " . $sql . "<br>" . $conn->error;
}

$conn->close();
?>
```

```
INSERT INTO table_name (column1, column2, column3,...)
VALUES (value1, value2, value3,...)
```

If a column is AUTO_INCREMENT (like the "id" column) or TIMESTAMP (like the "reg_date" column), it is no need to be specified in the SQL query; MySQL will automatically add the value.

Insert a data to "MyGuest" table

id	firstname	lastname	email	reg_date
1	John	Doe	john@example.com	10/01/2015

– Insert Multiple Records Into MySQL

```
$sql = "INSERT INTO MyGuests (firstname, lastname, email)
VALUES ('John', 'Doe', 'john@example.com');";
$sql .= "INSERT INTO MyGuests (firstname, lastname, email)
VALUES ('Mary', 'Moe', 'mary@example.com');";
$sql .= "INSERT INTO MyGuests (firstname, lastname, email)
VALUES ('Julie', 'Dooley', 'julie@example.com')";
```

- Concatenate the query string.
- Note that each SQL statement must be separated by a semicolon

```
if ($conn->multi_query($sql) === TRUE) {
    echo "New records created successfully";
} else {
    echo "Error: " . $sql . "<br>" . $conn->error;
}
```

Insert Multiple Records

– Prepared Statements and Bound Parameters

⇔ A prepared statement is a feature used to execute the same (or similar) SQL statements repeatedly with high efficiency.

```php
// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

// prepare and bind
$stmt = $conn->prepare("INSERT INTO MyGuests (firstname, lastname, email) VALUES (?, ?, ?)");
$stmt->bind_param("sss", $firstname, $lastname, $email);

// set parameters and execute
$firstname = "John";
$lastname = "Doe";
$email = "john@example.com";
$stmt->execute();

$firstname = "Mary";
$lastname = "Moe";
$email = "mary@example.com";
$stmt->execute();

$firstname = "Julie";
$lastname = "Dooley";
$email = "julie@example.com";
$stmt->execute();

echo "New records created successfully";

$stmt->close();
$conn->close();
?>
```

- Prepare and Bind
  - ✓ s: string,
  - ✓ i : integer,
  - ✓ d : double,
  - ✓ b : boolean

Insert Multiple Records efficiently

id	firstname	lastname	email	reg_date
1	John	Doe	john@example.com	2014-10-22 14:26:15
2	Mary	Moe	mary@example.com	2014-10-23 10:22:30
3	Julie	Dooley	julie@example.com	2014-10-26 10:48:23

- **Select Data From a MySQL Database**
  - The SELECT statement is used to select data from one or more tables:

```
SELECT column_name(s) FROM table_name
```

```
SELECT * FROM table_name
```

Select

Print

```
$sql = "SELECT id, firstname, lastname FROM MyGuests";
$result = $conn->query($sql);

if ($result->num_rows > 0) {
    // output data of each row
    while($row = $result->fetch_assoc()) {
        echo "id: " . $row["id"]. " - Name: " . $row["firstname"]. " " . $row["lastname"]. "<br>";
    }
} else {
    echo "0 results";
}
```

id	firstname	lastname	email	reg_date
1	John	Doe	john@example.com	2014-10-22 14:26:15
2	Mary	Moe	mary@example.com	2014-10-23 10:22:30
3	Julie	Dooley	julie@example.com	2014-10-26 10:48:23

Result:

id: 1 - Name: John Doe
id: 2 - Name: Mary Moe
id: 3 - Name: Julie Dooley

- **Delete Data From a MySQL Table**
  - The DELETE statement is used to delete records from a table:

```php
// sql to delete a record
$sql = "DELETE FROM MyGuests WHERE id=3";

if ($conn->query($sql) === TRUE) {
    echo "Record deleted successfully";
} else {
    echo "Error deleting record: " . $conn->error;
}
```

Delete record

id	firstname	lastname	email	reg_date
1	John	Doe	john@example.com	2014-10-22 14:26:15
2	Mary	Moe	mary@example.com	2014-10-23 10:22:30
3	Julie	Dooley	julie@example.com	2014-10-26 10:48:23

Before

id	firstname	lastname	email	reg_date
1	John	Doe	john@example.com	2014-10-22 14:26:15
2	Mary	Moe	mary@example.com	2014-10-23 10:22:30

After

- **Update Data In a MySQL Table**
  - Update : The WHERE clause specifies which record or records that should be updated. If you omit the WHERE clause, all records will be updated!

```
UPDATE table_name
SET column1=value, column2=value2,...
WHERE some_column=some_value
```

```
$sql = "UPDATE MyGuests SET lastname='Doe' WHERE id=2";

if ($conn->query($sql) === TRUE) {
    echo "Record updated successfully";
} else {
    echo "Error updating record: " . $conn->error;
}
```

Update record

id	firstname	lastname	email	reg_date
1	John	Doe	john@example.com	2014-10-22 14:26:15
2	Mary	Moe	mary@example.com	2014-10-23 10:22:30

Before

id	firstname	lastname	email	reg_date
1	John	Doe	john@example.com	2014-10-22 14:26:15
2	Mary	Doe	mary@example.com	2014-10-23 10:22:30

After

- **Limit Data Selections From a MySQL Database**
  - MySQL provides a LIMIT clause that is used to specify the number of records to return.
  - The LIMIT clause makes it easy to code multi page results or pagination with SQL, and is very useful on large tables.

    ⇔ Returning a large number of records can impact on performance.

```
$sql = "SELECT * FROM Orders LIMIT 30";
```

```
$sql = "SELECT * FROM Orders LIMIT 10 OFFSET 15";
```

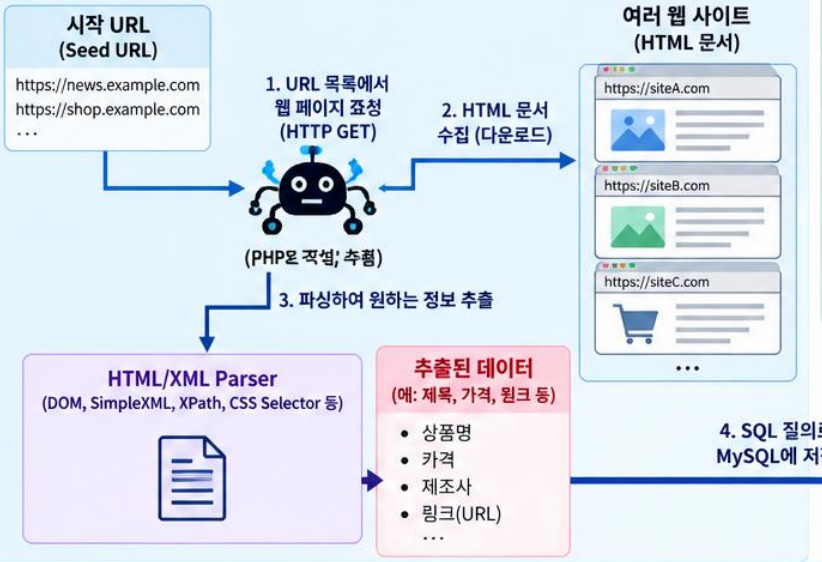
6. XML Parser and Web Crawling

Web Crawling을 하여 정보를 수집하는 이유와 방법

1 왜 Web Crawling을 하는가? (이유)

1. **다양한 시이트의 정보를 자동으로 수집**
   필요한 정보를 여러 웹 사이트에서
   일일이 방문하지 않고 자동으로 수집 가능
2. **최신 정보 확보**
   뉴스, 상품가격, 논문, 채용공고 등
   변경이 잦은 데이터를 주기적으로 수집하여
   항상 최신 상태로 유지
3. **대량 데이터를 효율적으로 수집**
   사람이 처리하기 어려운 방대한 웹 문서를
   자동으로 수집하여 분석 가능
4. **데이터 분석·서비스에 활용**
   수집한 데이터를 DB에 저장하여
   가격비교, 동향분석, 검색엔진, 추천서비스 등
   다양한 서비스 구현에 활용
5. **반복 작업의 자동화**
   정해진 규칙에 따라 주기적으로 실행되어
   지속적이고 일관된 데이터 수집 가능

2 Web Crawling의 기본 동작 원리



3 수집되는 정보의 예

웹 사이트 유형	수집 가능한 정보 예시
뉴스 사이트	기사 제목, 본문, 작성일, 기자, URL
쇼핑몰	상품명, 가격, 제조사, 이미지, 상세링크
채용 사이트	회사명, 채용공고, 근무조건, 마감일
학술 사이트	논문 제목, 저자, 초록, 발행일
부동산 사이트	매물 정보, 가격, 위치, 면적
블로그/커뮤니티	게시글 제목, 작성자, 내용, 댓글 수
...	...

4 Web Crawling 방법 (PHP 예제 흐름)

1 시작 URL 설정 (수집할 대상 사이트)

시작 URL 목록

```
https://news.example.com
https://shop.example.com
https://blog.example.com
...
```

수집할 사이트의 시작 URL을 목록으로 준비

2 웹 페이지 요청 (HTTP GET)

```php
<?php
 $url = 'https://siteA.com';
 $html = file_get_contents
        ($url, false, $content);
?>
```

- PHP의 file_get_contents(), cURL 등을 사용하여 웹 페이지의 HTML 문서 다운로드

3 HTML 문서 파싱 (원하는 정보 추출)

DOM / SimpleXML / XPath / CSS Selector

```php
<?php
 $dom = new DOMDocument();
 $dom->loadHTML($html);
 $items = $dom->getElementsBy
       ClassName('product');
 foreach ($items as $item) {
    $title = ...; // 제목 추출
    $price = ...; // 가격 추출
    $link  = ...; // 링크 추출
?>
```

4 추출된 데이터 가공 (필요한 항목 정리)

추출 데이터 (예시)

상품명	가격	URL
노트북	1,200,000원	/p1.html
카메라	850,000원	/p2.html
해드폰	200,000원	/p3.html
...	...	...

5 SQL 질의로 MySQL에 저장

```php
<?php
 $conn = new mysqli('localhost',
    'user', 'pass', 'testdb');
 $sql = "INSERT INTO products
        (title, price, url)
        VALUES (?, ?, ?)";
 $tmt = $conn->prepare($sql);
 $stmt->bind_param('sis',
        $title, $price, $url);
 $stmt->execute();
?>
```

5 Crawling 시 주의사항 (윤리 및 법적 고려)

- 사이트의 robots.txt 확인 (수집 허용 범위 준수)
- 이용약관(Terms of Service) 준수
- 과도한 요청으로 서버에 부하를 주지 않도록 요청 간격(Delay) 설정
- 저작권이 있는 콘텐츠의 무단 수집 · 사용 금지
- 개인정보 등 민감한 정보 수집 금지

robots.txt 예시

```
User-agent: *
Disallow: /private/
Disallow: /admin/
Allow: /
Crawl-delay: 10
```

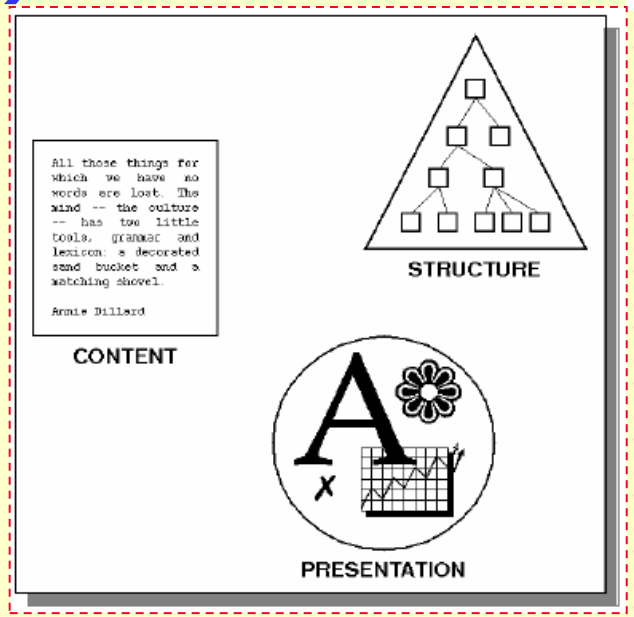
6 효율적인 Crawling을 위한 기법

- 큐(Queue)를 이용한 URL 관리 (BFS/DFS)
- 중복 URL 제거 (방문한 URL 집합 관리)
- 멀티 스레드/비동기 요청으로 속도 향상
- 페이지 변경 감지 (마지막 수정일, 해시값 등)
- 에러 처리 및 재시도 로직
- 주기적 스케줄링 (Cron 등)으로 자동 실행

7 수집 데이터 활용 예

- 가격 비교 사이트
- 뉴스/트렌드 분석 서비스
- 학술 논문 검색 시스템
- 채용 공고 알림 서비스
- 부동산 시세 분석
- 자체 검색엔진 구축
- 데이터 시각화 및 통계 분석

6.1 XML (eXtensible Markup Language)



- Document의 구성 요소
  - 내용 (Content)
  - 구조 (Structure)
  - 표현 (Presentation)
  - ➔ Document는 내용의 전달(표현) 및 저장 수단

- Markup 이란 ?
  - 문서의 표현이나 논리적 구조의 묘사와 같은 의미나 사용법 등의 추가적인 정보를 명시할 목적으로 특정 위치에 삽입되는 일련의 문자나 기호
  - 흔히 태그 (tag)라고 함
- Markup Language란 ?
  - 문서의 구조와 내용에 추가적인 의미를 부여하기 위하여 makrup 규칙을 규정하는 언어
  - 문서가 어떻게 "이해될 것인지"를 규정하는 문법과 구문 구조, 즉 makrup들을 규정하는 규칙
  - 예 : GML, SGML, HTML, XML, .. Latex, troff, nroff, …

6.1 XML (eXtensible Markup Language)

Evolution of Full-Text 관점에서 본 XML 문서의 등장 배경

1 단순 텍스트 시대 (순수 Full-Text)

~ 1980년대 초

- 컴퓨터에서 문서는 단순한 문자(Full-Text)의 나열로만 취급
- 내용만 있고 구조(의미) 정보는 없음
- 사람이 읽기는 쉽지만, 컴퓨터가 의미를 이해하기 어려움

```
Alice was beginning
to get very tired of
sitting by her sister
on the bank, and of
having nothing to do:
once or twice she had
peeped into the book
...
```

순수 텍스트 문서
(구조 정보 없음)

2 서식 있는 텍스트 (문서 표시 중심)

1980년대 중반 ~ 1990년대

- 문서를 화면에 보기 좋게 표자하기 위해 서식(포맷) 정보 추가
- 대표 형식: 워드프로세서 문서 (DOC, RTF 등)
- 내용과 서식이 혼합되어 있어, 컴퓨터가 문서의 의미 구조를 정확히 해석하기 어려움
- 시스템/프로그램마다 형식이 달라 호환성 문제 발생

```
[B]제목[/B]

이 문서는 [I]서식[/I]이 포함된
텍스트 문서입니다.

[FONT size=12 color=blue]
중요 내용
[/FONT]
```

워드프로세서 형식 예
(내용 + 서식이 혼합)

3 하이퍼텍스트의 등장 (HTML, 웹과 Full-Text)

1990년대 ~

- 웹에서 문서 간 연결(링크)을 제공하기 위해 HTML 등장
- 텍스트에 태그를 사용하여 간단한 구조(제목, 단락, 링크 등)를 표현
- 사람이 읽고 화면에 표시하는 데 적합
- 하지만 문서의 의미(데이터 구조)를 표현하는 데는 한계
  (태그의 의미가 고정되어 있고 확장성이 부족)

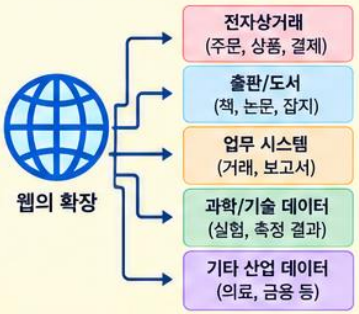
```
<html>
<head>
  <title>문서 제목</title>
</head>
<body>
  <h1>제목</h1>
  <p>이것은 단락입니다.</p>
  <a href="page.html">링크</a>
</body>
</html>
```

HTML 문서 예
(표시 중심의 제한된 구조)

4 더 다양한 문서, 더 많은 데이터 (웹, 전자상거래, 데이터 교환)

1990년대 후반

- 웹의 확산으로 문서의 종류와 데이터 교환 요구가 폭발적으로 증가
- 각 분야(전자상거래, 출판, 과학, 업무 시스템 등)마다 서로 다른 데이터 구조 필요
- HTML만으로는 다양한 도메인의 구조화된 데이터를 표현하기 어려움
- 사람뿐 아니라 컴퓨터도 이해할 수 있는 "의미가 있는 구조화된 문서 형식"이 요구됨



5 XML의 등장 (의미 기반의 구조화된 Full-Text)

2000년대 ~ (W3C 표준)

- 다양한 도메인의 문서를 공통적으로 표현하고 교환할 수 있는 확장 가능한 마크업 언어로 **XML** 등장
- 사용자가 원하는 태그를 적접 정의하여 데이터의 의미와 구조를 명확히 표현
- 사람이 읽기 쉽고, 컴퓨터가 처리하기 쉬운 텍스트 기반의 표준 형식
- 플랫폼/언어에 독립적이어서 시스템 간 데이터 교환에 적합
- 웹 서비스, 데이터 저장, 문서 관리 등 다양한 분야에서 널리 사용

```
<?xml version="1.0" encoding="UTF-8"?>
<book>
  <title>이상한 나라의 앨리스</title>
  <author>Lewis Carroll</author>
  <price>12000</price>
  <category>소설</category>
  <description>...
   </description>
</book>
```

XML 문서 예
(사용자 정의 태그로 의미와 구조를 표현)

왜 XML이 필요했는가? (Full-Text 관점 요약)

1. 단순한 텍스트만으로는 문서의 의미와 구조를 표현할 수 없었음
   → 컴퓨터가 데이터를 정확히 이해하고 처리하기 어려움
2. HTML은 표시(웹 페이지)에 특화되어 있어, 다양한 도메인의 데이터 구조를 표현하기에 한계
3. 웹의 확산으로 서로 다른 시스템 간에 데이터를 주고받아야 하는 요구 증가
   → 표준화된, 의미 기반의 문서 형식 필요
4. 플랫폼과 언어에 독립적으로 사용할 수 있는 확장 가능한 마크업 언어 필요
   → XML이 이러한 요구를 충족

Evolution 요약도 (Full-Text 관점)



텍스트만 존재 (의미, 구조 없음)	내용 + 서식 (의미 구조 불명확)	제한된 구조 (웹 문서에 특화)	분야별 데이터 구조 (표준 형식 필요)	사용자 정의 태그 확장 가능한 표준 형식

XML은 단순한 Full-Text를 넘어, 의미와 구조를 가진 범용적인 문서 형식으로 발전한 결과이다.

XML 형식의 문서가 필요한 이유와 예제

1 XML이 필요한 이유

1 데이터의 구조화 및 표준화
복잡한 데이터를 계층적 구조로 표현하여 누구나 이해할 수 있는 표준 형식 제공

2 서로 다른 시스템 간 데이터 교환
운영체제, 프로그래밍 언어, 몰랫품에 독립적으로 데이터를 주고받을 수 있음

3 데이터의 자기 설명성 (Self-describing)
태그를 사용하여 데이터의 의미와 구조를 스스로 설명 (사람과 컴퓨터 모두 이해 가능)

4 확장성과 유연성
필요에 따라 새로운 태그를 추가하여 데이터 구조를 쉽게 확장 가능

5 다양한 분야에서 활용
웹 서비스, 데이터 저장, 설정 파일, 문서 교환, RSS/Atom 피드, 사무 문서 등 다양한 분야에서 표준 형식으로 사용

6 사람과 기계가 모두 읽기 쉬움
텍스트 기반 형식으로 사람이 직접 읽고 편집할 수 있으며, 프로그램이 쉽게 처리 가능

2 XML 문서의 예 (쇼핑몰 상품 정보)

products.xml
(상품 목록 XML 문서)

```xml
<?xml version="1.0" encoding="UTF-8"?>
<products>
  <product>
    <id>1</id>
    <name>노트북</name>
    <price>1200000</price>
    <category>전자제품</category>
    <image>img1.jpg</image>
    <link>https://site.com/p1.html</link>
  </product>
  <product>
    <id>2</id>
    <name>카메라</name>
    <price>850000</price>
    <category>전자제품</category>
    <image>img2.jpg</image>
    <link>https://site.com/p2.html</link>
  </product>
  <product>
    <id>3</id>
    <name>헤드폰</name>
    <price>200000</price>
    <category>전자제품</category>
    <image>img3.jpg</image>
    <link>https://site.com/p3.html</link>
  </product>
</products>
```

3 XML 문서의 구조 설명

```xml
<?xml version="1.0" encoding="UTF-8"?>
<products>
  <product>
    <id>1</id>
    <name>노트북</name>
    <price>1200000</price>
    <category>전자제품</category>
    <image>img1.jpg</image>
    <link>https://site.com/p1.html</link>
  </product>
</products>
```

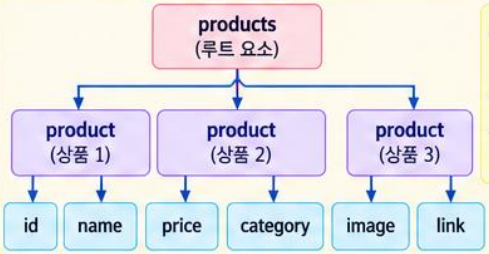
XML 선언문
- XML 버전과 인코딩을 지정
- 문서의 시작 부분에 위치

루트 요소 (Root Element)
- 전체 문서를 감싸는 최상위 태그
- 이 예에서는 <products>

자식 요소 (Child Element)
- 루트 요소 안에 포함되는 요소
- 반복적으로 여러 개 존재 가능 (여기서는 <product>)

데이터 요소
- 각 요소는 태그와 값으로 구성
- 태그 이름을 통해 데이터의 의미를 명확히 알 수 있음

XML 문서의 트리 구조 (계층 구조)



- XML은 계층적(트리) 구조로 되어 있어 복잡한 데이터를 구조적으로 표현 가능
- 각 요소는 부모-자식 관계를 가짐
- 이 구조를 이용하여 프로그램에서 필요한 데이터를 쉽게 추출할 수 있음

4 XML이 활용되는 주요 분야

웹 서비스 (데이터 교환)
서로 다른 시스템 간에 데이터를 주고받는 표준 형식 (SOAP, REST 응답 데이터 등)



RSS/Atom 피드
뉴스, 블로그 등의 최신 콘텐츠를 배포

```xml
<rss>
  <channel>
    <title>뉴스</title>
    <item>...</item>
  </channel>
</rss>
```

설정 파일
프로그램의 설정 정보를 XML로 저장

```xml
<config>
  <db>
    <host>localhost</host>
    <user>root</user>
    <password>1234</password>
  </db>
</config>
```

사무 문서 형식
오피스 문서도 내부적으로 XML 형식을 사용
(예: Microsoft Office의 .docx, .xlsx)

데이터 저장 및 전송
복잡한 데이터를 구조적으로 저장하고 전송



다양한 응용 분야
- 쇼핑몰 상품 정보
- 학술 논문 메타데이터
- 지도/지리 정보 (예: KML)
- 금융 데이터 (예: XBRL)
- 의료 데이터 (예: HL7)
- 설정/배포 파일 등

- **XML: used for what ?**
  - Its primary purpose is to facilitate the sharing of data across different systems, particularly systems connected via the Internet
  - Traditionally:
    - ⇔ DATA → encode in XML → sent → decode to own format
  - Nowadays data is often kept in XML format => XML DBMS
  - This gives rise to special XML technology

- **What is XML Good For ?**
  - XML allows users to define new collections of tags that can be *used to structure any type of data*. Examples are
    - ⇔ SVG for describing two-dimensional graphics,
    - ⇔ MathML for mathematical expressions, and
    - ⇔ XHTML, which is a reformulation of HTML4 in XML.
  - XML acts as a bridge between
    - ⇔ a presentation-oriented view of data (HMTL) and
    - ⇔ a schema-oriented view of data (DBMS).
  - XML is designed so that it is easy to write XML processors.

MathML : https://www.w3.org/Math/XSL/csmall2.xml
MusicML : https://www.w3.org/2021/06/musicxml40/musicxml-reference/examples/
SVG : https://www.w3schools.com/html/html5_svg.asp

XML의 X는 Extensible(확장 가능하다)이다

– 사용자가 새로운 태그와 구조를 정의하여, 목적에 맞는 언어(문서 형식)를 만들 수 있다 –

1 XML이 확장 가능한 이유

1. **태그 이름을 사용자가 자유롭게 정의할 수 있다.**
   XML은 미리 정해진 태그 집합이 없고,
   사용자가 목적에 맞게 새로운 태그를 적접 만들 수 있다.
2. **문서의 구조를 사용자가 설계할 수 있다.**
   필요한 데이터를 표현하기 위해 원하는 계층 구조와
   속성(attribute)을 정의할 수 있다.
3. **문법이 단순하고, 규칙이 유연하다.**
   기본적으로 <태그>...</태그> 형태만 지키면 되므로,
   특정 분야에 특화된 언어를 쉽게 설계할 수 있다.
4. **의미 기반의 마크업을 정의할 수 있다.**
   각 태그에 의미를 부여하여, 데이터의 구조와 의미를
   명확하게 표현할 수 있다.
5. **다른 기술과의 호환성이 높다.**
   XML은 텍스트 기반으로, 웹, 데이터베이스, 다양한
   프로그램 언어에서 쉽게 처리할 수 있다.

2 확장 가능성의 예 (사용자 정의 태그)

기본 XML 예 (사용자 정의 태그)

```
<?xml version="1.0" encoding="UTF-8"?>
<book>
  <title>XML의 이해</title>
  <author>홍길동</author>
  <price>25000</price>
</book>
```

사용자가 목적에 맞게 <book>, <title>, <author>, <price>와 같은 태그를 직접 정의하여 사용

새로운 태그를 추가하여 확장한 예

필요에 따라 새로운 태그를 자유롭게 추가 가능

```
<?xml version="1.0" encoding="UTF-8"?>
<book>
  <title>XML의 이해</title>
  <author>홍길동</author>
  <price>25000</price>
  <category>컴퓨터</category>        <!-- 새로운 태그 추가 -->
  <publisher>정보출판사</publisher> <!-- 새로운 태그 추가 -->
</book>
```

3 확장성을 이용한 다양한 XML 언어

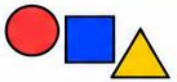
XML의 확장 기능을 이용하여, 각 분야에 특화된 XML 기반의 언어(문서 형식)를 만들 수 있다.

MusicXML (음악 분야)



악보 데이터를 표현하는 XML 언어

SVG (그래픽 분야)



2차원 벡터 그래픽을 표현하는 XML 언어

MathML (수학 분야)

$$\int_0^1 x^2\,dx$$

수학 공식을 표현하는 XML 언어

DocBook (문서 출판 분야)



기술 문서, 전자책 등을 작성하는 XML 언어

4 예제 1 : MusicXML (악보 데이터를 표현하는 XML 언어)

MusicXML 문서 예 (일부)

```
<?xml version="1.0" encoding="UTF-8"?>
<score-partwise version="4.0">
  <work>
    <work-title>Twinkle Twinkle Little Star</work-title>
  </work>
  <part-list>
    <score-part id="P1">
      <part-name>Piano</part-name>
    </score-part>
  </part-list>
  <part id="P1">
    <measure number="1">
      <note>
        <pitch>
          <step>C</step>
          <octave>4</octave>
        </pitch>
        <duration>1</duration>
        <type>quarter</type>
      </note>
      ...
    </measure>
  </part>
</score-partwise>
```

위 MusicXML을 해석하여 표시한 악보 (예)



주요 태그의 의미

태그	설명
<work-title>	곡의 제목
<part>	연주 파트 (예: Piano, Violin 등)
<measure>	마디 정보
<note>	음표 하나의 정보
<pitch>	음의 높이 (계이름, 옥타브)
<duration>	음의 길이
<type>	음표의 종류 (quarter, half 등)

5 예제 2 : SVG (2차원 벡터 그래픽을 표현하는 XML 언어)

SVG 문서 예 (간단한 도형)

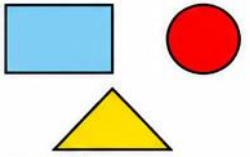
```
<?xml version="1.0" encoding="UTF-8"?>
<svg width="300" height="200"
     wmlns="http://www.w3.org/2000/svg">
  <rect x="10" y="10" width="100" height="60"
        fill="skyblue" stroke="black" stroke-width="2"/>

  <circle cx="180" cy="40" r="30"
        fill="red" stroke="black" stroke-width="2"/>

  <polygon points="50,150 100,100 150,150"
        fill="gold" stroke="black" stroke-width="2"/>

  <text x="10" y="190" font-size="16">SVG 예제</text>
</svg>
```

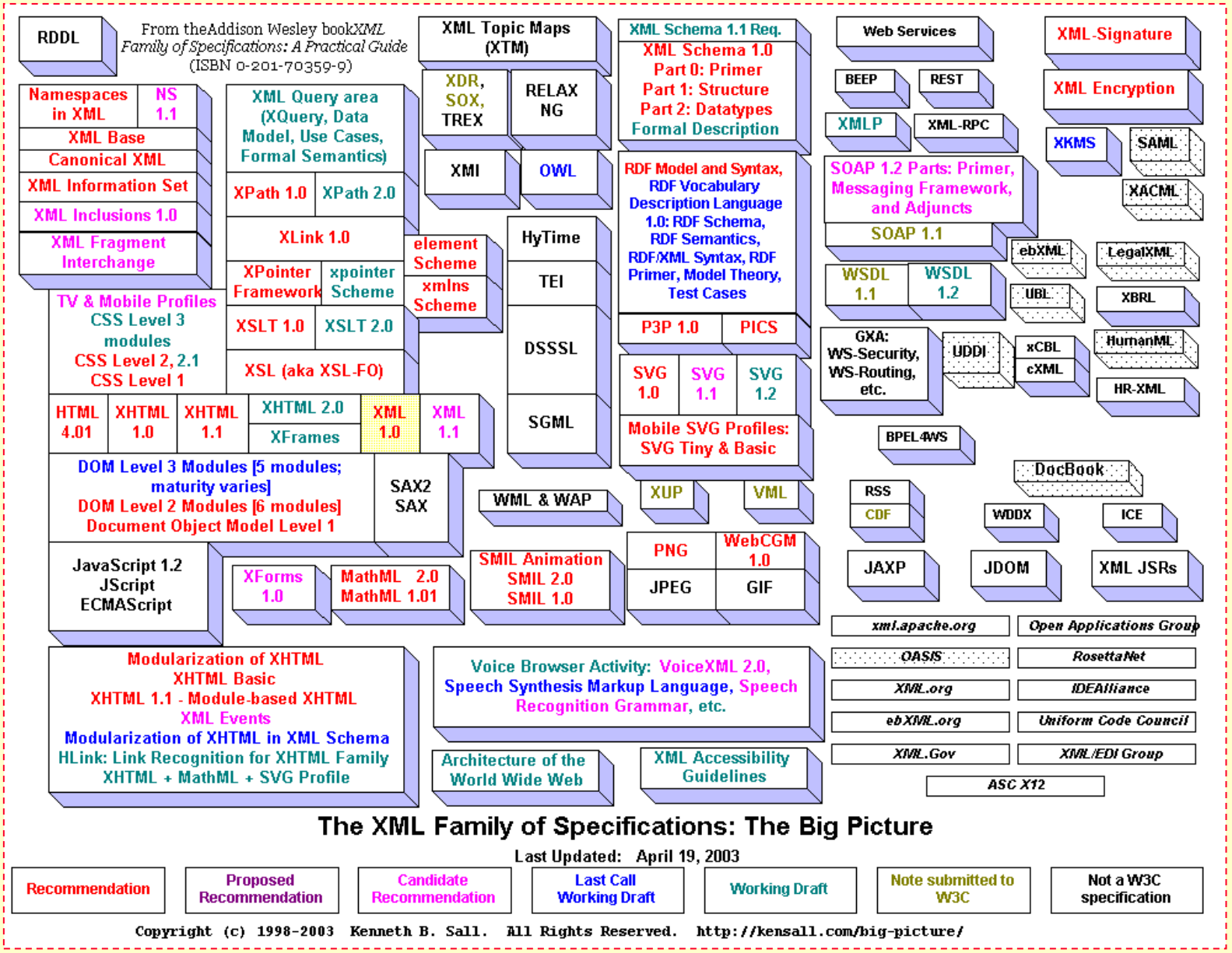
위 SVG를 브라우저에서 표시한 결과



주요 태그의 의미

태그	설명	예시 의미
<svg>	SVG 문서의 루트 태그	전체 그래픽 영역 ㅁ
<rect>	사각형	x, y, width, height, 색성
<circle>	원	중심 좌표(cx, cy), 반지름(r)
<polygon>	다각형	여러 정의 좌표로 도형 생성
<text>	텍스트	화면에 텍스트 표시

- **A Small Sample of XML-related Languages**

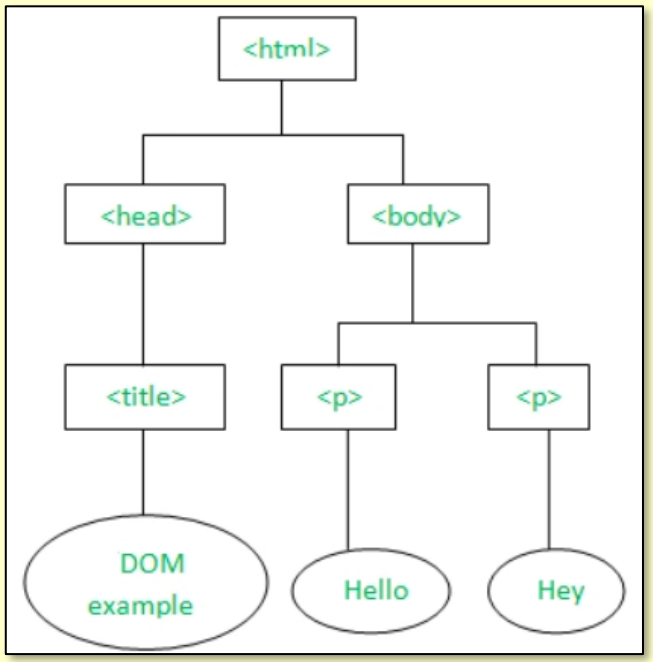


6.2 XML 문서 Parsing using PHP DOM Parser

- **DOM (Document Object Model) 이란 ?**
  - 문서 객체 모델(The Document Object Model, 이하 DOM) 은 HTML, XML 문서의 프로그래밍 interface 이다.
  - DOM은 문서의 구조화된 표현(structured representation)을 제공하며 프로그래밍 언어가 DOM 구조에 접근할 수 있는 방법을 제공하여 그들이 문서 구조, 스타일, 내용 등을 변경할 수 있게 돕는다
  - DOM은 트리 구조로 HTML 문서를 표현한다.

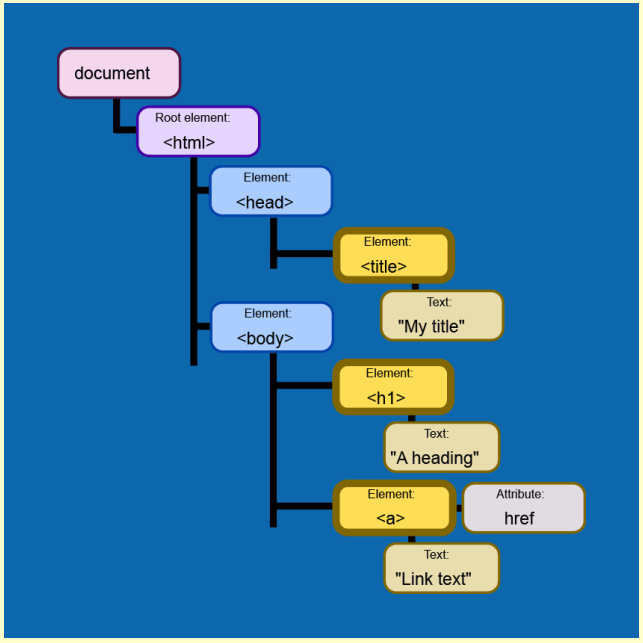
```
<html>
<head>
<title>
 DOM example
</title>
</head>
<body>
<p id = "para1">Hello</p>
<p id = “para2”>Hey</p>
</body>
</html>
```

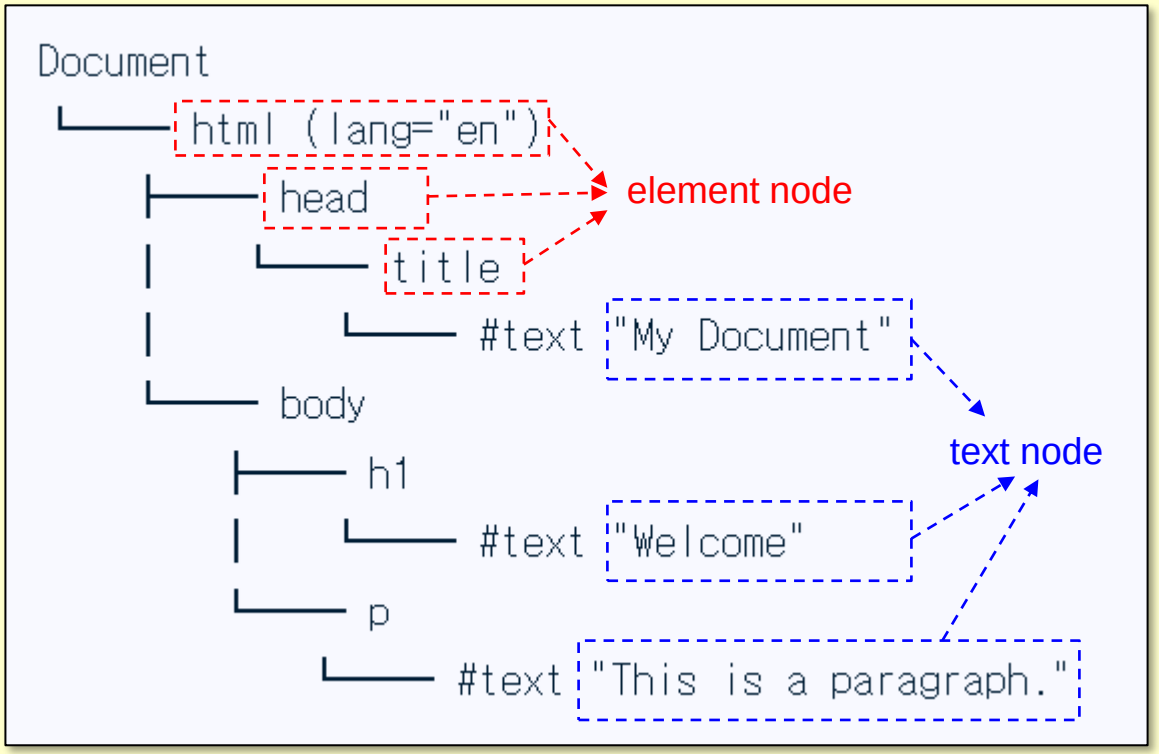
HTML 문서



DOM Tree

```
<!DOCTYPE html>
<html lang="en">
<head>
    <title>My Document</title>
</head>
<body>
    <h1>Welcome</h1>
    <p>This is a paragraph.</p>
</body>
</html>
```



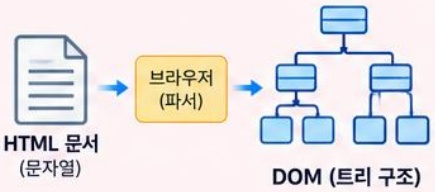


DOM (Document Object Model) 이란?

HTML, XML 문서를 트리 구조로 매모리에 표현하여 프로그래밍 언어로 문서의 구조와 내용을 탐색하고, 수정할 수 있는 인터페이스

1 DOM의 개념

- 웹 브라우저가 HTML/XML 문서를 읽어 트리 구조의 객체 구조로 메모리에 표현한 것
- 각 태그, 속성, 텍스트를 '**노드(Node)**'로 표현
- 프로그래밍 언어를 이용하여 노드를 탐색, 생성, 수정, 삭제 가능
- 문서의 구조와 내용을 쉽게 조작할 수 있도록 제공하는 표준 인터페이스 (W3C DOM 표준)

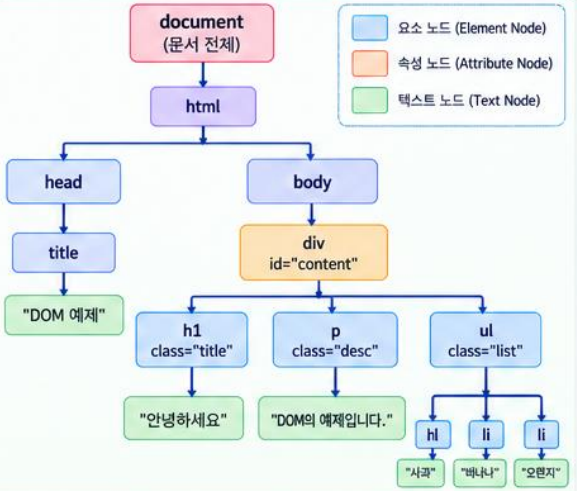


2 HTML 문서 예제

```html
<!DOCTYPE html>
<html>
 <head>
   <title>DOM 예제</title>
 </head>
 <body>
   <div id="content">
     <h1 class="title">안녕하세요</h1>
     <p class="desc">DOM의 예제입니다.</p>
     <ul class="list">
       <li>사과</li>
       <li>바나나</li>
       <li>오렌지</li>
     </ul>
   </div>
 </body>
</html>
```

HTML 문서 (텍스트 형태)

3 DOM 트리 구조



DOM 트리 구조 (메모리 상의 객체 구조)

4 DOM의 주요 노드 종류

요소 노드 (Element Node)

- HTML/XML의 태그에 해당하는 노드
- 예) , <p>, <ul> 등
- 노드 이름(nodeName)으로 태그 이름을 가짐

속성 노드 (Attribute Node)

- 태그의 속성에 해당하는 노드
- 예) id="content", class="title"
- 요소 노드의 attributes 컬렉션으로 접근

텍스트 노드 (Text Node)

- 태그 내부의 실제 텍스트 내용
- 예) "안녕하세요", "사과" 등
- nodeType이 3이고, nodeValue로 값 접근

기타 노드

- 문서 노드 (Document Node) : 전체 문서
- 주석 노드 (Comment Node) : <!-- 주석 -->
- 문서 타입 노드 (DocumentType Node) : <!DOCTYPE ...>

5 DOM을 이용한 문서 탐색 방법 (예시)

```javascript
// 1. 요소 찾기 (ID로)
const content = document.getElementById('content');

// 2. 클래스로 찾기
const title = document.getElementsByClassName('title')[0];

// 3. 태그 이름으로 찾기
const lis = document.getElementsByTagName('li');

// 4. CSS 선택자로 찾기 (더 유연한 방법)
const desc = document.querySelector('.desc');
const firstLi = document.querySelector('#content ul li');

// 5. 관계 노드 탐색
const parent = title.parentNode;  // 부모 노드 (div)
const children = content.children; // 자식 요소 노드들
const next = title.nextElementSibling;  // 다음 형제 노드 (p)
```

6 DOM을 이용한 문서 조작 방법 (예시)

1 노드 내용 변경

```javascript
const title = document.querySelector('.title');
title.textContent = '안녕하세요!';
title.setAttribute('class', 'main-title');
```

2 새 노드 생성 및 추가

```javascript
// 새로운 목록 항목 생성
const li = document.createElement("li");
li.textContent = "포도";
// 기존 <ul>에 추가
const ul = document.querySelector('.list");
ul.appendChild(li);
```

3 노드 삭제

```javascript
const ul = document.querySelector('.list');
const items = ul.getElementsByTagName('li');
// 첫 번째 항목 삭제
ul.removeChild(items[0]);
```

4 속성 값 읽기

```javascript
const link = document.querySelector('a');
const href = link.getAttribute("href");
console.log(href);
```

DOM의 활용 예

- JavaScript로 웹 페이지의 내용을 동적으로 변경 (예: 클릭 시 내용 바꾸기)
- 웹 크롤링 시 HTML 문서를 파싱하여 필요한 정보 추출
- XML 문서를 읽고 데이터 처리
- 프로그래밍으로 새로운 HTML/XML 문서를 생성

핵심 정리

- DOM은 문서를 트리 구조의 객체로 표현한 모델이다.
- 각 태그, 속성, 텍스트는 노드로 표현된다.
- 프로그래밍 언어를 통해 노드를 탐색하고, 내용을 조회·수정·추가·삭제할 수 있다.
- 웹 브라우저, 웹 크롤링, XML 처리 등 다양한 분야에서 널리 사용된다.

- **DOM API**
  - XML 문서를 구성하는 각각의 요소를 객체로 모델링화 함으로써 객체가 가진 '속성'과 '메소드'를 통해 문서의 구조와 데이터를 다룰 수 있도록 적의한 표준화된 프로그래밍 인터페이스 ➔ 프로그래밍 언어나 운영체제에 중립적이고 독립적이다.
  - W3C가 국제표준으로 제안한 XML 프로그래밍 인터페이스)
  - 예제 프로그램

DOMDocument class : https://www.php.net/manual/en/class.domdocument.php
DOMElement class : https://www.php.net/manual/en/class.domelement.php

```php
$xmlToLoad = <<<XML
<?xml version="1.0" encoding="utf-8"?>
<library>
<magazine title="Tech Weekly" issue="123"/>
<magazine title="Science Today" issue="456"/>
</library>
XML;

$domLoad = new DOMDocument();
$domLoad->loadXML($xmlToLoad);

echo "\nLoaded XML:\n";
echo $domLoad->saveXML();

// Find elements by tag name
$magazines = $domLoad->getElementsByTagName('magazine');
echo "\nMagazine Titles:\n";
foreach ($magazines as $magazine) {
echo $magazine->getAttribute('title') . "\n";
}
```

- **PHP Simple HTML DOM Parser** (https://simplehtmldom.sourceforge.io/docs/1.9/index.html)

Finding elements by tag name

```
// Find all anchors, returns a array of element objects
$ret = $html->find('a');

// Find all anchors and images, returns an array of element objects
$ret = $html->find('a, img');

// Find (N)th anchor, returns element object or null if not found (zero based)
$ret = $html->find('a', 0);

// Find last anchor, returns element object or null if not found (zero based)
$ret = $html->find('a', -1);
```

Finding elements by class name or id

```
// Find all element which id=foo
$ret = $html->find('#foo');

// Find all element which class=foo
$ret = $html->find('.foo');
```

Finding elements by attribute

```
// Find all <div> with the id attribute
$ret = $html->find('div[id]');

// Find all <div> which attribute id=foo
$ret = $html->find('div[id=foo]');

// Find all anchors and images with the "title" attribute
$ret = $html->find('a[title], img[title]');

// Find all element has attribute id
$ret = $html->find('*[id]');
```

https://simplehtmldom.sourceforge.io/docs/1.9/manual/finding-html-elements/

Attribute filters

Supports these operators in attribute selectors:

Filter	Description
`[attribute]`	Matches elements that **have** the specified attribute.
`[!attribute]`	Matches elements that **don't have** the specified attribute.
`[attribute=value]`	Matches elements that have the specified attribute with a **certain value**.
`[attribute!=value]`	Matches elements that **don't have** the specified attribute with a certain value.
`[attribute^=value]`	Matches elements that have the specified attribute and it **starts** with a certain value.
`[attribute$=value]`	Matches elements that have the specified attribute and it **ends** with a certain value.
`[attribute*=value]`	Matches elements that have the specified attribute and it **contains** a certain value.

Finding descendants

```php
// Find all <li> in <ul>
$es = $html->find('ul li');

// Find Nested <div> tags
$es = $html->find('div div div');

// Find all <td> in <table> which class=hello
$es = $html->find('table.hello td');

// Find all td tags with attribite align=center in table tags
$es = $html->find('table td[align=center]');
```

Finding nested elements

```php
// Find all <li> in <ul>
foreach($html->find('ul') as $ul)
{
	foreach($ul->find('li') as $li)
	{
		// do something...
	}
}

// Find first <li> in first <ul>
$e = $html->find('ul', 0)->find('li', 0);
```

Finding text blocks and comments

```php
// Find all text blocks
$es = $html->find('text');

// Find all comment (<!--...-->) blocks
$es = $html->find('comment');
```

PHP DOM 파서를 이용한 XML 문서 Parsing 및 정보 추출

1 PHP DOM 파서란?

- PHP의 DOM (Document Object Model) 확장 모듈을 사용하여 XML 문서를 트리 구조로 메모리에 로드하고, 노드(Node)를 탐색하여 원하는 정보를 추출하는 방법
- W3C DOM 표준을 따르며, 객체지향 방식으로 XML 문서를 다룸
- 원본 XML 문서를 변경(추가, 수정, 삭제)할 수도 있음
- 복잡한 구조의 XML 문서도 손쉽게 탐색 가능

DOM 방식의 처리 개념도



2 예제 XML 문서 (책 정보)

books.xml (예제 XML 문서)

```xml
<?xml version="1.0" encoding="UTF-8"?>
<bookstore>
    <book id="b1">
        <title>XML 완벽 가이드</title>
        <author>홍길동</author>
        <price>25000</price>
        <category>컴퓨터</category>
    </book>
    <book id="b2">
        <title>PHP 실무</title>
        <author>김영희</author>
        <price>22000</price>
        <category>웹 개발</category>
    </book>
    <book id="b3">
        <title>데이터베이스 개론</title>
        <author>이철수</author>
        <price>30000</price>
        <category>데이터베이스</category>
    </book>
</bookstore>
```

3 PHP DOM 파서를 이용한 XML 파싱 예제

parse_xml.php (XML 파싱 및 정보 추출)

```php
<?php
 // 1. DOMDocument 객체 생성
 $doc = new DOMDocument();

 // 2. XML 파일 로드
 $doc->load("books.xml");

 // 3. 루트 요소 가져오기 (<bookstore>)
 $root = $doc->documentElement;

 // 4. 모든 <book> 요소 가져오기
 $books = $root->getElementsByTagName("book");

 // 5. 각 <book> 요소를 순회하며 정보 추출
 foreach ($books as $book) {
    $id = $book->getAttribute("id");   // 속성값
    $title = $book->getElementsByTagName("title")->item(0)->textContent;
    $author = $book->getElementsByTagName("author")->item(0)->textContent;
    $price = $book->getElementsByTagName("price")->item(0)->textContent;
    $category = $book->getElementsByTagName("category")->item(0)->textContent;

    // 결과 출력 (또는 데이터베이스 저장 등 원하는 처리)
    echo "ID: $id | 제목: $title | 저자: $author | 가격: $price | 분야: $category<br>";
 }
?>
```

주요 DOM 메서드

- new DOMDocument() : DOM 객체 생성
- load() : XML 파일 로드
- documentElement : 루트 요소
- getElementsByTagName() : 태그 검색
- item(0) : 첫 번째 요소 반환
- getAttribute() : 속성값 가져오기
- textContent : 요소 내 텍스트 얻기

4 실행 결과

웹 브라우저 출력 결과 (예시)

ID: b1 | 제목: XML 완벽 가이드 | 저자: 홍길동 | 가격: 25000 | 분야: 컴퓨터

ID: b2 | 제목: PHP 실무 | 저자: 김영희 | 가격: 22000 | 분야: 웹 개발

ID: b3 | 제목: 데이터베이스 개론 | 저자: 이철수 | 가격: 30000 | 분야: 데이터베이스

5 특정 조건의 정보 추출 예제

가격이 25,000원 이상인 책만 추출하여 출력

```php
<?php
 $doc = new DOMDocument();
 $doc->load("books.xml");
 $books = $doc->getElementsByTagName("book");

 foreach ($books as $book) {
    $price = (int)$book->getElementsByTagName("price")
                ->item(0)->textContent;
    if ($price >= 25000) {
        $title = $book->getElementsByTagName("title")
                    ->item(0)->textContent;
        echo "[25,000원 이상] $title ($price원)<br>";
    }
 }
?>
```

6 다른 활용 예

1 XML 문서 수정 및 저장

새로운 책 정보를 추가한 후 XML 파일로 저장

```php
<?php
 $doc = new DOMDocument();
 $doc->load("books.xml");

 $book = $doc->createElement("book");
 $book->setAttribute("id", "b4");
 $book->appendChild($doc->createElement(
      "title", "자바의 정석"));
 $book->appendChild($doc->createElement(
      "author", "남궁성"));
 $book->appendChild($doc->createElement(
      "price", "28000"));
 $book->appendChild($doc->createElement(
      "category", "프로그래밍"));

 $doc->documentElement->appendChild($book);
 $doc->save("books_updated.xml");
?>
```

2 XPath를 이용한 정보 추출 (간단 예제)

특정 분야(카테고리)의 책만 추출

```php
<?php
$doc = new DOMDocument();
$doc->load("books.xml");
$xpath = new DOMXPath($doc);

$nodes = $xpath->query("/bookstore/
          book[category='컴퓨터']");
foreach ($nodes as $book) {
   $title = $book->getElementsByTagName
     ("title")->item(0)->textContent;
   echo "[컴퓨터 분야] $title<br>";
}
?>
```

HTML 문서를 Parsing하여 원하는 정보를 추출하는 과정 (예제: 쇼핑몰 상품 정보 수집)

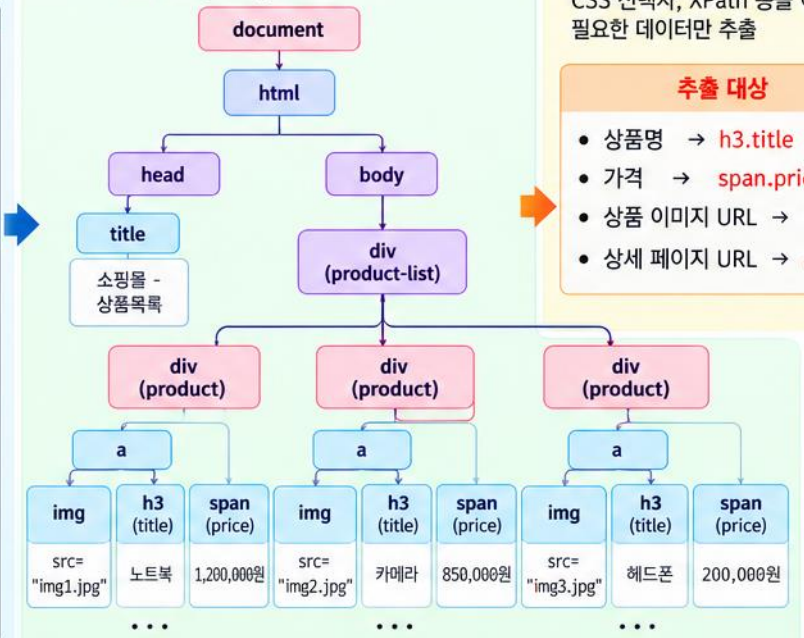
1 HTML 문서 (원본 소스)

웹 페이지에서 다운로드한 HTML 문서의 일부 예시

```
<html>
 <head>
  <title>쇼핑몰 - 상품목록</title>
 </head>
 <body>
  <div class="product-list">
   <div class="product">
    <a href="/p1.html">
     <img src="img1.jpg" alt="노트북">
     <h3 class="title">노트북</h3>
     <span class="price">1,200,000원</span>
    </a>
   </div>
   <div class="product">
    <a href="/p2.html">
     <img src="img2.jpg" alt="카메라">
     <h3 class="title">카메라</h3>
     <span class="price">850,000원</span>
    </a>
   </div>
   <div class="product">
    <a href="/p3.html">
     <img src="img3.jpg" alt="헤드폰">
     <h3 class="title">헤드폰</h3>
     <span class="price">200,000원</span>
    </a>
   </div>
  ..div>
 </body>
</html>
```

2 HTML 파싱 (Parsing)

HTML Parser가 문서를 해석하여 트리 구조(DOM)로 변환



3 원하는 정보 추출 방법

DOM 트리에서 특정 태그, 클래스, CSS 선택자, XPath 등을 이용하여 필요한 데이터만 추출

추출 대상

- 상품명 → h3.title
- 가격 → span.price
- 상품 이미지 URL → img[src]
- 상세 페이지 URL → a[href]

4 파싱 결과 (추출된 데이터)

원하는 정보만 배열/객체 형태로 추출

상품명	가격	이미지 URL	상품 페이지 URL
노트북	1,200,000원	img1.jpg	/p1.html
카메라	850,000원	img2.jpg	/p2.html
헤드폰	200,000원	img3.jpg	/p3.html
...	...	...	...

5 데이터 저장 (예: MySQL)

추출한 데이터를 SQL을 사용하여 데이터베이스에 저장

```
INSERT INTO products (title, price, image_url, page_url)
VALUES ('노트북', 1200000, 'img1.jpg', '/p1.html'),
       ('카메라', 850000,  'img2.jpg', '/p2.html');
       ('헤드폰', 200000,  'img3.jpg', '/p3.html');
```

주요 HTML 파싱 방법 (PHP 예제)

A DOM 방식 (DOMDocument, DOMXPath)

- 장점 : 표준 방식, 구조적 탐색 가능, 복잡한 문서에 적합
- 단점 : 코드가 다소 길 수 있음

```
<?php
$html = file_get_contents('https://example.com/products.html');
$dom = new DOMDocument();
@$dom->loadHTML($html);
$xpath = new DOMXPath($dom);

$nodes = $xpath->query('//div[@class="product"]');
foreach ($nodes as $node) {
    $title = $node->getElementsByTagName('h3')->item(0)->textContent;
    $price = $node->getElementsByTagName('span')->item(0)->textContent;
    $img   = $node->getElementsByTagName('img')->item(0)->getAttribute('src');
    $url   = $node->getElementsByTagName('a')->item(0)->getAttribute('href');
    echo "$title / $price / $img / $url\n";
}
?>
```

B SimpleXML 방식

- 장점 : 간단한 문서에 대해 사용여 쉽고 코드가 짧음
- 단점 : 잘못된 HTML에는 동작하지 않을 수 있음

```
<?php
$html = file_get_contents('https://example.com/products.html');
$xml = simplexml_load_string($html, 'SimpleXMLElement',
        LIBXML_NOCDATA | LIBXML_HTML_NODEFDTD);

foreach ($xml->xpath('//div[@class="product"]') as $item) {
    $title = (string) $item->xpath('.//h3')[0];
    $price = (string) $item->xpath('.//span')[0];
    $img   = (string) $item->xpath('.//img/@src')[0];
    $url   = (string) $item->xpath('.//a/@href')[0];
    echo "$title / $price / $img / $url\n";
}
?>
```

C CSS 선택자 방식 (Querylist)

- 장점 : CSS 선택자로 간단하고 직관적인 추출 가능
- 단점 : 외부 라이브러리 설치 필요 (composer)

```
# Composer로 QueryList 설치 : composer require jaeger/querylist
<?php
require 'vendor/autoload.php';
use QL\QueryList;
#html = file_get_contents('https://example.com/products.html');
$ql = QueryList::html($html);
$items = $ql->find('div.product');

foreach ($items as $item) {
    $title = $item->find('h3.title')->text();
    $price = $item->find('span.price')->text();
    $img   = $item->find('img')->attr('src');
    $url   = $item->find('a')->attr('href');
    echo "$title / $price / $img / $url\n";
}
?>
```

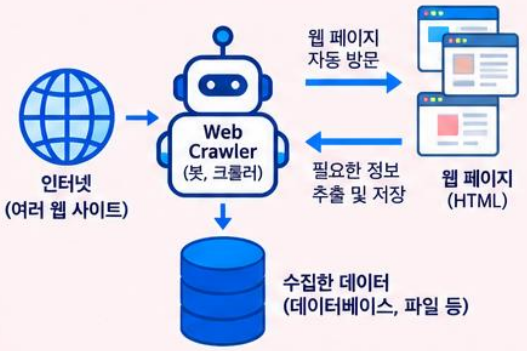
6.3 Web Crawling

Web Crawling 이란?

웹 상의 여러 웹 페이지를 자동으로 방문하여 필요한 정보를 수집하는 프로그램(봇, 크롤러)의 동작 방식입니다.

1 Web Crawling 이란?

- 웹 브라우저가 사람이 직접 웹 페이지를 방문하여 내용을 보는 것과 유사하게,
  프로그램(크롤러, 봇, Robot)이 자동으로
  여러 웹 사이트를 방문하여 페이지를 가져오고,
  필요한 정보를 추출하는 기술입니다.



2 동작 과정 (개념도)

1. **시작 URL 등록**
   크롤링을 시작할 웹 페이지 주소를 지정
   (예: https://example.com)
2. **웹 페이지 다운로드**
   HTTP 요청을 보내 HTML 문서를 가져옴
3. **링크(<a> 태그) 추출**
   페이지 내의 <a> 태그에서 새로운 URL을 추출

```
<a href="https://site.com/page.html">링크</a>
```

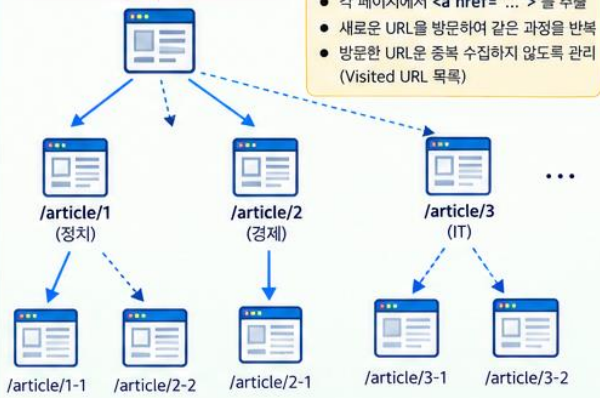
4. **필요한 정보 추출**
   원하는 데이터(제목, 가격, 내용 등)를 파싱하여 저장
5. **새로운 URL 반복 방문**
   추출한 링크를 큐에 추가하고, 방문하지 않은
   새 페이지를 계속 수집 (종료 조건까지 반복)

3 여러 페이지를 방문하는 예시 흐름

시작 URL
https://news.example.com/

<a> 태그를 따라 링크를 발견하고 새로운 페이지를 계속 방문

- 각 페이지에서 <a href="..."> 를 추출
- 새로운 URL을 방문하여 같은 과정을 반복
- 방문한 URL은 중복 수집하지 않도록 관리 (Visited URL 목록)



4 Web Crawling이 필요한 이유

1. **대량의 정보를 빠르게 수집**
   - 사람이 일일이 방문하기 어려운 수많은 페이지를 자동으로 수집
   - 뉴스, 상품, 논문, 채용 공고 등 최신 데이터 수집 가능
2. **데이터 분석 및 서비스에 활용**
   - 수집한 데이터를 분석하여 시장 동향, 여론 분석, 가격 비교, 트렌드 파악 등에 활용
   - 검색엔진, 추천 시스템, 가격 비교 사이트 등 다양한 서비스의 기반 데이터로 사용
3. **반복적이고 정기적인 정보 수집 자동화**
   - 매일 업데이트되는 정보를 정해진 시간에 자동으로 수집
   - 수집 결과를 데이터베이스에 저장하여 지속적으로 활용
4. **비즈니스 의사결정 지원**
   - 경쟁사 가격 모니터링, 신상품 정보 수집, 고객 리뷰 분석 등
   - 빠르고 정확한 데이터 수집을 통해 더 나은 의사결정 가능

5 간단한 Web Crawling 예제 (PHP + DOM Parser)

대상 웹 페이지 (예: 도서 목록)

https://example.com/books.html

```
<ul class="book-list">
  <li>
    <a href="/book/1.html">PHP 완벽 가이드</a>
    <span class="price">25,000원</span>
  </li>

  <li>
    <a href="/book/2.html">웹 프로그래밍</a>
    <span class="price">22,000원</spann>
  </li>
</ul>

...
```

PHP 크롤러 예제 코드

```php
<?php
$url = "https://example.com/books.html";
$html = file_get_contents($url); // HTML 가져오기
$dom = new DOMDocument();
@$dom->loadHTML($html);
$xpath = new DOMXPath($dom);

$nodes = $xpath->query('//ul[@class="book-list"]/li');
foreach ($nodes as $node) {
  $title = $xpath->query('.//a', $node)->item(0)->textContent;
  $link = $xpath->query('.//a', $node)->item(0)->getAttribute('href');
  $price = $xpath->query('.//span[@class="price"]', $node)->item(0)->
                                                    textContent;
  echo "제목: $title | 링크: $link | 가격: $price\n";
}
```

실행 결과 (일부)

```
제목: PHP 완벽 가이드 | 링크: /book/1.html | 가격: 25,000원
제목: 웹 프로그래밍  | 링크: /book/2.html | 가격: 22,000원
...
```

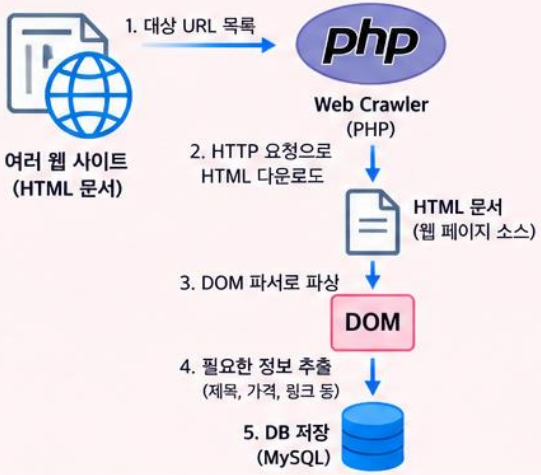
6 활용 분야 예시

- **검색엔진**
  웹을 크롤링하여 수집한 문서를 색인(index)으로 저장하고 검색 서비스 제공 (예: Google, Naver)
- **가격 비교 서비스**
  여러 쇼핑몰의 상품 정보와 가격을 수집하여 비교 (예: 다나와, 에누리)
- **뉴스 및 콘텐츠 수집**
  다양한 언론사의 뉴스를 수집하여 분석 (예: 뉴스 모니터링, 미디어 분석)
- **시장/트렌드 분석**
  특정 키워드, 상품, 산업의 최신 동향 파악 (예: 주식, 부동산, 소비 트렌드)
- **학술 및 연구 데이터 수집**
  논문, 특허, 연구 자료 등 학술 정보를 수집 (예: 논문 검색, 연구 동향 분석)
- **기타 자동화 업무**
  채용 공고 수집, 리뷰 수집, 지도 정보 수집 등 다양한 비즈니스 자동화에 활용

PHP DOM Parser를 이용한 Web Crawling 방법 (예제 + 프로그램 소스 포함)

웹 페이지를 수집하여 HTML을 DOM 파서로 분석하고, 필요한 정보를 추출하여 데이터베이스에 저장하는 전체 과정을 예제로 설명합니다.

1 전체 처리 흐름도



2 크롤링 대상 웹 페이지 예시

(예: 도서 상품 목록 페이지)

https://example.com/books.html

```
<li class="book-list">
 <li class="book">
   <a href="/book/1.html"><img src="img1.jpg"></a>
   <h2 class="title">PHP 완벽 가이드</h2>
   <span class="author">홍길동</span>
   <span class="price">25,000원</span>
 </>
 <li class="book">
   <a href="/book/2.html"><img src="img2.jpg"></a>
   <h2 class="title">웹 프로그래밍</h2>
   <span class="author">김영희</span>
   <span class="price">22,000원</span>
 </>
                ...
```

3 DOM 파서로 HTML 파싱

DOMDocument로 HTML을 읽어 DOM 트리 구조로 변환

DOM 트리(일부)

- document
  - html
    - body
      - ul (class=book-list)
        - li (class=book)
          - a (href=/book/1.html)
            - img (src=img1.jpg)
          - h2 (class=title) PHP 완벽 가이드
          - span (class=author) 홍길동
          - span (class=price) 25,000원
        - ...

4 필요한 정보 추출 (예시)

DOM 노드를 탐색하여 원하는 데이터 추출

추출할 정보

- 도서 제목 (h2.title)
- 저자 (span.author)
- 가격 (span.price)
- 상세 페이지 링크 (a href)
- 이미지 추소 (img src)

추출 결과 (배열 형태)

```
Array (
   [title] => PHP 완벽 가이드
   [author] => 홍길동
   [price] => 25000
   [link] => https://example.com/book/1.html
   [image] => https://example.com/img1.jpg
)
   ...
```

5 PHP 웹 크롤러 + DOM 파서 예제 소스 코드

상품 목록 페이지를 크롤링하여 도서 정보를 추출하고, MySQL에 저장하는 PHP 예제입니다.

```php
<?php
// 크롤링할 시작 URL (도서 목록 페이지)
$listUrl = "https://example.com/books.html";

// 1. HTML 가져오기
$html = @file_get_contents($listUrl);
if ($html === false) {
    die("페이지를 가져올 수 없습니다.");
}

// 2. DOMDocument로 파싱
$dom = new DOMDocument();
libxml_use_internal_errors(true); // HTML 파싱 오류 무시
$dom->loadHTML($html);
libxml_clear_errors();

// 3. DOMXPath 생성 (CSS 선택자처럼 쿼리 가능)
$xpath = new DOMXPath($dom);

// 4. 각 도서 정보 추출
$books = [];
$nodes = $xpath->query("//ul[contains(@class,"book-list")]/li[contains(@class,"book")]");
foreach ($nodes as $node) {
    $title  = trim($xpath->query('.//h2[@class="title"]', $node)->item(0)->textContent);
    $author = trim($xpath->query('.//span[@class="author"]', $node)->item(0)->textContent);
    $price  = trim(str_replace('원', '', $xpath->query('.//span[@class="]', $node)->item(0)->textContent));
    $link   = $xpath->query('.//a', $node)->item(0)->getAttribute('href');
    $image  = $xpath->query('.//img', $node)->item(0)->getAttribute('src');

    // 상대 URL을 절대 URL로 변환
    $base = 'https://example.com';
    if (strpos($link, 'http') !== 0)  $link  = $base . $link;
    if (strpos($image, 'http') !== 0) $image = $base . $image;
    $books[] = [
        'title'  => $title,
        'author' => $aathor,
        'price'  => (int)$price,
        'link'   => $link,
        'image'  => $image
    ];
}
```

```php
// 5. MySQL에 저장
$host = "localhost";
$user = "root";
$pass = "1234";
$db   = "books";
$conn = new mysqli($host, $user, $pass, $db);
if ($conn->connect_error) {
    die("DB 연결 실패: " . $conn->connect_error);
}
// 테이블이 없으면 생성
$conn->query("CREATE TABLE IF NOT EXISTS book (
    id INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(200),
    author VARCHAR(100),
    price INT,
    link VARCHAR(500),
    image VARCHAR(500),
    crawled_at DATETIME DEFAULT CURRENT_TIMESTAMP
)");
// 추출한 데이터 저장
$stmt = $conn->prepare("INSERT INTO book (
(title, author, price, link, image) VALUES (?, ?, ?, ?, ?)");
foreach ($books as $book) {
    $stmt->bind_param("ssisss",
        $book['title'],
        $book['author'],
        $book['price'],
        $book['link'],
        $book['image'],
    );
    $stmt->execute();
}
echo "총 " . count($books) . "개의 도서 정보를 저장했습니다.\n";

$conn->close();
?>
```

6 실행 결과 예시

추출한 도서 정보 (화면 출력)

총 3개의 도서 정보를 저장했습니다.

1. PHP 완벽 가이드 (홍길동) - 25000원
   링크: https://example.com/book/1.html
   이미지: https://example.com/img1.jpg
2. 웹 프로그래밍 (김영희) - 22000원
   링크: https://example.com/book/2.html
   이미지: https://example.com/img2.jpg

...

MySQL 저장 결과 (일부)

id	title	author	price
1	PHP 완벽 가이드	홍길동	25000
2	웹 프로그래밍	김영희	22000
3	데이터베이스 개론	이철수	30000
...	...	...	...

7 확장 예시

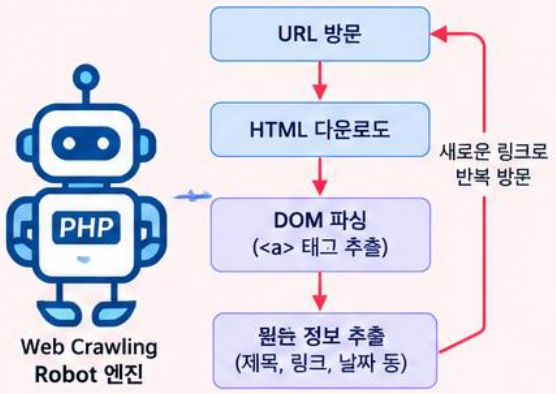
- 상세 페이지까지 크롤링하여 책 소개, 출판사, 출간일 등 추가 정보 수집
- 페이지네이션이 있는 경우 다음 페이지 자동 수집
- 중복 저장 방지 (이미 저장된 링크는 제외)
- 수집한 데이터를 JSON, CSV, XML 등 다양한 형식으로 저장
- 이미지 파일도 서버에 다운로드하여 관리
- 로그 기록, 예외 처리, 크롤링 간격 설정 (과도한 요청 방지)

<a> 태그를 따라 여러 사이트를 방문하며 정보를 수집하는 Web Crawling Robot 엔진 (PHP DOM Parser)

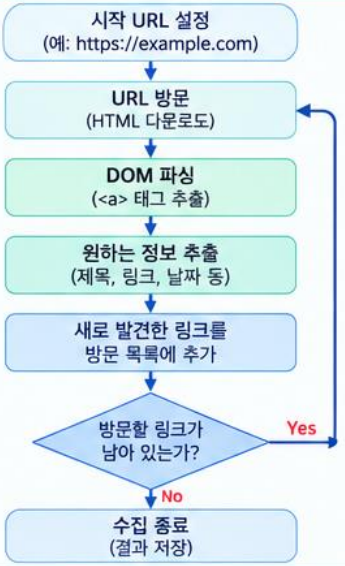
시작 URL에서 <a> 태그의 링크를 추출하여, 새로운 페이지를 반복적으로 방문하면서, 원하는 정보를 자동으로 수집하는 로봇(크롤러) 엔진의 개념과 예제 프로그램

1 Robot 엔진의 개념

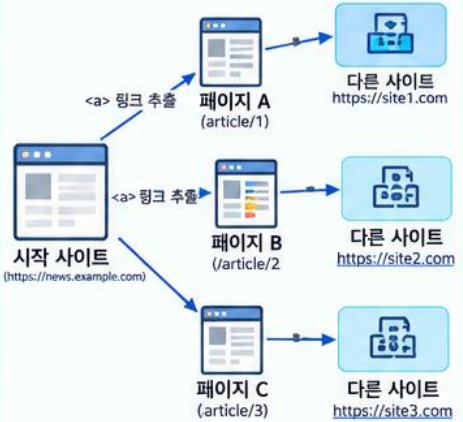
웹 페이지의 <a> 태그 링크를 추출하여,
새로운 페이지를 반복적으로 방문하면서
필요한 정보를 자동으로 수집하는 프로그램.



2 동작 흐름도 (Flow Chart)



3 여러 사이트를 방문하는 예시



작 페이지에서 <a> 태그의 href 링크를 추출하여,
같은 도메인 또는 다른 도메인의 페이지도 계속 방문할 수 있습니다.
(조건에 따라 특정 도메인만 방문하도록 제한 가능)

4 수집 가능한 정보의 예

```
<div class="news-list">
   <a href="https://site1.com/a1.html">
      <h2>새로운 기술 동향 발표</h2>
      <span class="date">2025-06-01</span>
      <img src="img1.jpg" />
   </a>
   <a href="https://site2.com/b1.html">
      <h2>AI 관련 기사</h2>
      <span class="date">2025-06-02</span>
      <img src="img2.jpg" />
   </a>
</div>
```

추출 데이터 (예)

- 제목 : 새로운 기술 동향 발표
- 링크 : https://site1.com/a1.html
- 날짜 : 2025-06-01
- 이미지 : https://site1.com/img1.jpg
- ...

5 PHP DOM Parser를 이용한 간단한 크롤러 예제

```php
<?php
// 시작 URL과 설정
$startUrl = 'https://news.example.com';
$maxPages = 10;          // 최대 방문 페이지 수
$visited = [];           // 이미 방문한 URL 저장
$queue = [$startUrl];    // 방문할 URL 큐
$results = [];           // 수집한 데이터 저장

while (!empty($queue) && count($visited) < $mazPages) {
    $url = array_shift($queue);
    if (isset($visited[$url])) continue;  // 이미 방문한 URL이면 건너뜀
    echo "방문 중: $url\n";
    $visited[$url] = true;

    // 1. HTML 가져오기
    $html = @file_get_contents($url);
    if ($html === false) continue;

    // 2. DOM 파서로 파싱
    $dom = new DOMDocument();
    @dom->loadHTML($html);
    $xpath = new DOMXPath($dom);
```

```php
    // 3. 원하는 정보 추출 (예: 뉴스 목록)
    $articles = $xpath->query("//div[contains(@class,'news-list')]//a");
    foreach ($articles as $a) {
        $titleNode = $xpath->query(".//h2", $a)->item(0);
        $dateNode = $xpath->query(".//span[contains(@class,'date')]", $a)->item(0);
        $dmgNode  = $xpath->query(".//img", $a)->item(0);
        $link = $a->getAttribute('href');
        $title = $titleNode ? trim($titleNode->textContent) : '';
        $date = $dateNode ? trim($dateNode->textContent) : '';
        $img  = $imgNode ? $imgNode->getAttribute('src') : '';
        $results[] = [
            'title' => $title,
            'link'  => $link,
            'date'  => $date,
            'image' => $img
        ];
        // 4. 새로운 링크를 큐에 추가 (절대/상대 URL 처리)
        if (!empty($link)) {
            $nextUrl = $link;
            // 상대 URL 처리 (간단한 예)
            if (strpos($nextUrl, 'http') !== 0) {
                $nextUrl = rtrim($url, '/') . '/' . ltrim($nextUrl, '/');
            }
            if (!isset($visited[$nextUrl])) {
                $queue[] = $nextUrl;
            }
        }
    }
?>
```

6 실행 결과 예시

```
방문 중: https://news.example.com
방문 중: https://news.example.com/article/1
방문 중: https://site1.com/a1.html
방문 중: https://news.example.com/article/2
방문 중: https://site2.com/b1.html
... (설정한 최대 페이지 수만큼 반복 방문)
```

수집된 데이터 (일부 출력 예)

제목	링크	날짜	이미지
새로운 기술 동향 발표	https://site1.com/a1.html	2025-06-01	img1.jpg
AI 관련 기사	https://site2.com/b1.html	2025-06-02	img2.jpg
웹 개발 트렌드	https://site3.com/c1.html	2025-06-03	img3.jpg
...	...	...	...

7 추가 고려 사항

- 특정 도메인만 방문하도록 필터링 (예: example.com만)
- robots.txt 확인 및 크롤링 에의 준수 (요청 간 시간 지연)
- 중복 URL 방지 (이미 방문한 URL 저장)
- 수집한 데이터를 파일(JSON, CSV) 또는 데이터베이스에 저장
- 더 복잡한 사이트는 CSS 선택자, XPath 등으로 추출 규칙을 조정